

推荐算法岗

推荐系统面试 100 题

从召回、排序、多目标到生成式推荐

定位

面向推荐算法 / 搜推算法 / 广告算法岗位的系统化面试手册

问题驱动 · 逐题拆解 · 公式推导 · 工程权衡 · 连续追问

2026 Edition · Professional Interview Handbook

使用说明

这不是一本“背答案”的题库，而是一套面试推理框架。结构参考优秀技术面试书常用的“问题驱动—思路拆解—追问递进”方法，但所有文字、结构和版式均为重新设计。公开面经只用于确认问题真实性与趋势，不代表任何公司官方题库。

三类来源标记：“面经高频”表示在公开面经中直接或高度同义出现；“论文追问”表示由经典模型题自然展开的常见追问；“系统设计”表示从项目深挖与工业链路中归纳出的高概率开放题。

推荐用法

第一遍：只看“30秒回答”；第二遍：掌握深入拆解和公式；第三遍：遮住正文，只用连续追问自测。面试前一周重点刷★★★★。

知识地图

层级	核心主题	代表模型 / 方法	典型指标
召回	高覆盖、低延迟、候选互补	ItemCF / Two-Tower / ANN / InfoNCE	Recall@K / HitRate / ANN Recall
粗排/精排	特征交叉、序列兴趣、多目标	DeepFM / DCNV2 / DIN / DIEN / PLE	AUC / GAUC / LogLoss
重排	列表级价值与约束	MMR / Quota / Diversity / Exploration	NDCG / Coverage / Long-term Utility
数据与实验	偏差、在线一致性、因果验证	IPS / Time Split / A-B	Lift / CI / SRM / Guardrails
前沿	Scaling 与生成式建模	HSTU / RankMixer / Semantic ID	Quality-Cost-Latency Pareto

目录

建议先按章节掌握主线，再回到题号进行高频冲刺。PDF 书签同时支持按章节与题目跳转。

第1章 系统架构与业务理解	6
01. 请完整介绍一个工业级推荐系统的链路	7
02. 召回、粗排、精排、重排分别解决什么问题?	8
03. 为什么推荐系统要多路召回?	8
04. 推荐系统为什么不能只优化 CTR?	9
05. 如何从零设计一个短视频推荐系统?	9
06. 推荐系统里的北极星指标应该怎么选?	10
07. 推荐系统如何做用户分层?	10
08. 搜索和推荐最大的区别是什么?	11
09. 如果推荐系统 CTR 突然下降 20%，怎么排查?	11
10. 如何评价一个推荐项目做得好不好?	12
第2章 召回、Embedding、双塔与负采样	13
11. UserCF 和 ItemCF 有什么区别?	14
12. ItemCF 有哪些经典改进?	15
13. 什么是矩阵分解? 为什么能做推荐?	15
14. 双塔模型为什么适合大规模召回?	16
15. 双塔最大的缺点是什么? 如何改?	16
16. 双塔通常怎么训练? InfoNCE 在做什么?	17
17. 为什么 In-batch Negative 很有效?	17
18. In-batch Negative 有什么严重问题?	18
19. Random Negative、Hard Negative、Semi-hard Negative 有什么区别?	18
20. 曝光未点击 Item 能不能直接当负样本?	19
21. InfoNCE 的 temperature 有什么作用?	19
22. 为什么热门 Item 不能无限作为负样本?	20
23. Embedding 维度应该怎么选?	20
24. 如何验证召回 Embedding 学得好不好?	21
25. ANN 为什么能解决大规模向量召回? HNSW、IVF、PQ 怎么选?	21
第3章 CTR 精排与特征交叉	23
26. 为什么推荐排序需要特征交叉?	24
27. FM 为什么适合推荐系统?	25
28. Wide & Deep 的核心思想是什么?	25
29. DeepFM 与 Wide & Deep 最大区别是什么?	26
30. DeepFM 的 FM 二阶项怎么高效计算?	26
31. DCN 为什么比纯 MLP 更适合特征交叉?	27

32. DCNv2 改进在哪里?	27
33. XGBoost 和 LightGBM 有什么区别?	28
34. 稀疏特征和 Dense Feature 处理方式有什么不同?	28
35. 连续特征应该标准化还是分桶?	29
36. 标准化应该在 Embedding 拼接前还是拼接后?	29
37. 如何做特征选择与特征重要度分析?	30
38. 离线 AUC 提升, 为什么线上可能没效果?	30
第4章 用户行为序列与长兴趣建模	32
39. DIN 为什么出现? 它解决了什么问题?	33
40. DIN 的 Attention 具体怎么计算?	34
41. DIN 和 Self-Attention 有什么区别?	34
42. DIN 有什么缺点?	35
43. DIEN 相比 DIN 改进在哪里?	35
44. DIEN 为什么需要 Auxiliary Loss?	36
45. GRU、AGRU、AUGRU 有什么区别?	36
46. 为什么用户行为序列需要去重、去噪和辅助信息?	37
47. 超长用户序列为什么不能直接上普通 Transformer?	37
48. SIM 如何处理超长行为序列?	38
49. Transformer 为什么适合用户行为序列?	38
50. 为什么推荐 Transformer 常用 LayerNorm 而不是 BatchNorm?	39
第5章 多任务、多目标与 CVR 建模	40
51. 为什么推荐系统需要 Multi-Task Learning?	41
52. Shared Bottom 有什么问题?	42
53. MMoE 怎么解决任务冲突?	42
54. MMoE 和普通 MoE 最大区别是什么?	43
55. 什么叫多任务“跷跷板现象”?	43
56. PLE 相比 MMoE 改进在哪里?	44
57. 多任务 Loss 怎么加权?	44
58. 一个任务收敛了, 另一个还没收敛怎么办?	45
59. CTR、CVR 为什么不能直接独立训练?	45
60. ESMM 的核心原理是什么?	46
61. ESMM 有什么不足? 后续模型为什么继续改?	46
62. 连续目标(如观看时长)应该怎么建模?	47
第6章 Loss、排序指标与在线实验	48
63. 为什么 CTR 常用 Binary Cross Entropy?	49
64. Pointwise、Pairwise、Listwise 有什么区别?	50
65. AUC 的本质是什么?	50
66. 正负样本严重不平衡时 AUC 为什么仍较稳定?	51

67. AUC 有什么缺点? 为什么推荐常看 GAUC?	51
68. GAUC 具体怎么算? 权重怎么选?	52
69. Precision、Recall、F1 在推荐中怎么用?	52
70. NDCG 为什么适合推荐排序?	53
71. 离线评价为什么必须重视 Time Split?	53
72. A/B Test 如何分桶?	54
73. 为什么 A/B 前最好做 A/A?	54
74. CTR 显著上涨但停留时长下降, 怎么办?	55
75. 线上提升 0.1% 到底算不算有效?	55
第7章 偏差、数据与在线工程	56
76. 推荐数据有哪些常见 Bias?	57
77. Position Bias 怎么解决?	58
78. IPS 是什么? 有什么问题?	58
79. 怎么解决类别不平衡?	59
80. 为什么训练样本不能随便随机负采样?	59
81. 什么是 Train-Serving Skew?	60
82. 在线训练和离线训练有什么区别?	60
83. 如何解决数据分布漂移?	61
84. 推荐模型线上延迟过高怎么优化?	61
85. Embedding 表太大怎么办?	62
第8章 冷启动、多样性、长尾与探索	63
86. 什么是 User Cold Start? 怎么做?	64
87. Item Cold Start 怎么做?	65
88. User Cold Start 和 Item Cold Start 最大区别是什么?	65
89. 如何解决热门 Item 吞噬长尾 Item?	66
90. 怎么评价推荐多样性?	66
91. 如何在准确率和多样性之间 trade-off?	67
92. 如何解决“信息茧房”?	67
93. 为什么新内容需要“保量”? 如何保量?	68
第9章 2026 前沿: HSTU、RankMixer 与生成式推荐	69
94. HSTU 为什么可能比普通 Transformer 更适合推荐?	70
95. 什么是 Generative Recommendation?	70
96. 生成式召回和双塔召回有什么区别?	71
97. 为什么生成式推荐常需要 Semantic ID / RQ-VAE?	71
98. RankMixer 想解决什么问题?	72
99. LLM 怎么进入推荐系统?	72
100. 如果重新设计下一代推荐系统, 你选传统 Pipeline 还是生成式推荐?	73

系统架构与业务理解

先把推荐系统当作一个受 QPS、延迟、数据闭环和业务目标约束的工程系统。面试官常从“完整链路”切入，再沿着你的项目逐层追问。

面试趋势

代表面经：美团/快手反复出现“完整链路、各阶段角色、多路召回、用户分层、项目是否落地”。

第 1 章 · 复习要点

01. 请完整介绍一个工业级推荐系统的链路
02. 召回、粗排、精排、重排分别解决什么问题？
03. 为什么推荐系统要多路召回？
04. 推荐系统为什么不能只优化 CTR？
05. 如何从零设计一个短视频推荐系统？
06. 推荐系统里的北极星指标应该怎么选？
07. 推荐系统如何做用户分层？
08. 搜索和推荐最大的区别是什么？
09. 如果推荐系统 CTR 突然下降 20%，怎么排查？
10. 如何评价一个推荐项目做得好不好？

01 请完整介绍一个工业级推荐系统的链路

★★★

面经高频

30 秒回答

核心回答

工业级推荐不是单模型，而是“数据闭环 + 多阶段检索排序 + 在线服务 + 实验平台”的系统。典型在线链路是多路召回→粗排→精排→重排/混排；离线侧同时维护样本、特征、Embedding、索引、训练和评估，并由曝光/点击/停留/转化日志回流形成闭环。

深入拆解

召回追求高覆盖，在数亿到数十亿物料里快速缩小到千级候选；粗排用很小计算预算砍到百级；精排使用最丰富特征和复杂多目标模型；重排处理列表级多样性、业务规则和生态约束。

真正的工程重点是 train-serving consistency：离线特征、线上特征、模型版本、ANN 索引与样本口径必须一致。

高分回答要主动补充 SLA/QPS/特征新鲜度/灰度发布/A-B 实验/监控，而不是只画四个框。

公式 / 工程抓手

KEY

$$N_item: 10^8 \sim 10^{10} \rightarrow Recall: 10^3 \sim 10^4 \rightarrow Pre_rank: 10^2 \sim 10^3 \rightarrow Rank: 10^1 \sim 10^2$$

常见失分

只背“召回-排序-重排”四个名词，不解释各层为什么存在、计算预算和指标差异。

连续追问

1. 为什么不能直接精排全库？
2. 召回和排序分别看什么离线指标？
3. 哪个阶段最容易造成线上 latency 尖峰？

02

召回、粗排、精排、重排分别解决什么问题？

★★★

面经高频

30 秒回答

核心回答

四阶段本质是“效果与计算预算”的分层优化。越靠前越强调覆盖率和吞吐，越靠后越强调精确性与列表整体价值。

深入拆解

召回：高 Recall、低延迟，可容忍较多噪声；粗排：用低成本模型筛掉明显不合适候选；精排：强交叉、长序列、多目标；重排：从 point-wise 转向 list-wise，处理多样性、去重、作者/类目打散、商业约束。

各层候选分布不同，因此不能直接拿精排 AUC 评价召回，也不能拿召回 Recall@K 评价最终列表满意度。

工业上常增加“轻量精排/粗排蒸馏/多阶段重排”，边界取决于业务延迟预算。

公式 / 工程抓手

KEY

目标可概括为：前层 maximize coverage / cost，后层 maximize utility / constraint。

常见失分

把重排理解成“再跑一次精排”。重排通常需要列表级状态和约束。

连续追问

1. 粗排为什么越来越重要？
2. 重排如何处理多个业务目标？
3. 阶段之间是否应该共享模型？

03

为什么推荐系统要多路召回？

★★★

面经高频

30 秒回答

核心回答

单一召回回路只能覆盖一种相似性假设；真实用户兴趣是多兴趣、多时间尺度、多意图，因此用 ItemCF、双塔、关注关系、热门、新鲜、地域等多路并集提高候选覆盖，再由后续排序统一比较。

深入拆解

不同召回回路应追求“互补”而不是简单堆数量。应评估每一路的独立 Hit、去掉该路后的边际损失，以及候选重叠率。

配额不是固定真理：可以按用户活跃度、兴趣熵、场景、流量阶段动态分配；也可以学习 routing/gating。

热门召回保证稳定性，新鲜召回解决 item cold start，协同/向量召回负责个性化，社交/地理召回引入结构先验。

公式 / 工程抓手

KEY

$$C = C_CF \cup C_TwoTower \cup C_Hot \cup C_Follow \cup C_Fresh; \Delta_i = Metric(A11) - Metric(A11-i)$$

常见失分

只说“多路能提高召回率”，不讨论冗余、配额和边际增益。

连续追问

1. 多路候选如何去重和融合？
2. 某一路 Recall 高但上线无收益怎么办？
3. 用户兴趣很窄时是否还需要多路？

04

推荐系统为什么不能只优化 CTR?

★★★

面经高频

30 秒回答

核心回答

CTR 是短期代理指标，不等于用户价值。只优化点击容易诱导标题党、浅层刺激和过度热门，导致完播、时长、满意度或留存下降。工业推荐通常是多目标优化，并对长期指标和负反馈设置 guardrail。

深入拆解

目标选择应从业务北极星指标反推：短视频关注有效观看时长/留存，电商关注成交/长期购买价值，社区还要考虑创作者生态。

模型训练的多任务概率与最终线上 utility 不是一回事，线上常需要价值函数、校准和约束。

短期代理与长期价值可能存在延迟和因果混淆，需要 A/B 长周期观察。

公式 / 工程抓手

KEY

$$U = \alpha \cdot \text{CTR} + \beta \cdot \text{WatchTime} + \gamma \cdot \text{CVR} + \delta \cdot \text{Retention} - \lambda \cdot \text{NegativeFeedback}$$

常见失分

把多目标等同于“把几个 loss 加起来”，忽略目标口径、校准和线上组合。

连续追问

1. 多目标权重怎么定？
2. CTR 上升而时长下降怎么办？
3. 长期留存无法快速得到 label 怎么办？

05

如何从零设计一个短视频推荐系统？

★★★

系统设计

30 秒回答

核心回答

先定义用户价值与约束，再搭建多阶段漏斗。召回可覆盖协同、向量、关注、热门、地域和新内容；精排同时预测点击、有效播放、完播、点赞、评论、转发、负反馈；重排处理作者/类目多样性、探索与安全规则。

深入拆解

不要从“用 DIN 还是 Transformer”开始；先问日活、物料规模、QPS、时延、用户行为类型、冷启动比例。

短视频的观看时长和视频长度强相关，必须做 duration bias 处理；完播率也会偏向短视频。

新内容需要探索流量，否则“无曝光→无样本→低分→更无曝光”形成闭环。

公式 / 工程抓手

KEY

最终排序通常是 calibrated multi-task scores 的非线性组合，而非单一 CTR。

常见失分

只给模型清单，没有数据链路、实验、冷启动、时长偏差和列表约束。

连续追问

1. 视频长度偏差怎么处理？

2. 新作者如何保量？
3. 如何防止推荐越来越窄？

06

推荐系统里的北极星指标应该怎么选？

★★★

系统设计

30 秒回答

核心回答	北极星指标必须尽可能接近长期业务价值，同时可被实验稳定测量。模型 label 只是代理；例如短视频可用有效观看时长或活跃天数，电商可用成交/复购，社区还需兼顾消费和供给生态。
------	---

深入拆解

- 指标体系至少分成主指标、护栏指标和诊断指标。主指标决定是否上线，护栏防止伤害用户或生态，诊断指标解释变化来源。
- 北极星指标若太远，会造成反馈延迟；太近，则容易被模型投机。实际需要多时间尺度指标。
- 不同用户生命周期可有不同价值函数，但最终必须能汇总到统一实验决策。

公式 / 工程抓手

KEY	North Star $\neq$ Training Label；更像长期价值函数的可观测代理。
常见失分	把 CTR/GMV 机械地当成所有业务的唯一北极星指标。

连续追问

1. 如果主指标不显著但多个代理指标都涨怎么办？
2. 如何设计护栏指标？
3. 生态指标怎么进入实验决策？

07

推荐系统如何做用户分层？

★★

面经高频

30 秒回答

核心回答	用户分层不是简单的人群报表，而是让召回、特征、模型或探索策略适应不同数据密度与生命周期。常见维度包括新老用户、活跃度、兴趣稳定性、行为长度、价值阶段和场景。
------	--

深入拆解

- 新用户更多依赖内容、上下文、热门与探索；高活跃老用户可以充分使用长序列、多兴趣召回和实时兴趣。
- 分层可以规则化，也可以由模型学习 gating；关键是验证分层确实存在异质性，而不是人为增加复杂度。
- 需要避免“分层标签泄漏”和过度固定用户画像，动态行为应能快速改变策略。

公式 / 工程抓手

KEY	策略可写成 $\pi(a u,segment,context)$ ，但 segment 应作为条件而非永久身份。
常见失分	把用户分层只用于报表，不解释它如何改变模型或策略。

连续追问

1. 用户分层和多任务/多域建模有什么关系？
2. 如何验证分层有效？
3. 新老用户是否应该完全两套模型？

08

搜索和推荐最大的区别是什么？

★★

面经高频

30 秒回答

核心回答

搜索有显式 Query，第一约束是相关性；推荐通常没有显式 Query，需要从用户历史与上下文推断当前意图。两者都需要检索、排序和多目标优化，现代“搜广推”正在共享越来越多表示与基础设施。

深入拆解

搜索 ranking 通常先保证 relevance，再优化点击/转化；推荐需要同时解决“想要什么”和“给什么”。

推荐里的历史行为可看作隐式 Query；搜索也会加入用户画像、session 和个性化，因此边界逐渐模糊。

生成式检索、统一 embedding、多任务排序正在推动统一架构，但线上约束仍不同。

公式 / 工程抓手

KEY

Search: $\text{score}(q,u,i,c)$; Rec: $\text{score}(u,\text{history},i,c)$ ，其中 history 承担隐式意图。

常见失分

回答成“搜索主动、推荐被动”就结束，缺少建模和指标层面的区别。

连续追问

1. 搜索相关性和 CTR 冲突怎么办？
2. 能否用同一个双塔服务搜索和推荐？
3. Query Rewrite 对推荐有何对应物？

09

如果推荐系统 CTR 突然下降 20%，怎么排查？

★★★

系统设计

30 秒回答

核心回答

先按“数据→流量→服务→特征→候选→模型→策略→实验”分层排查，优先验证是否为埋点、流量结构或服务故障，而不是第一时间归因模型。

深入拆解

检查埋点与 denominator 是否变化、实验分桶是否 SRM、请求成功率和时延是否异常、特征缺失率与 freshness、ANN 索引版本、召回量与分数分布。

做时间维度、用户维度、场景维度、模型版本维度切片，定位异常从何时/何群体开始。

若所有模型同时掉，通常先怀疑数据或流量；只有新模型掉，则优先查模型/特征/serving skew。

公式 / 工程抓手

KEY

排障核心：找到最早发生分布断点的层，而不是盯最终 CTR。

常见失分

“回滚模型”是应急动作，不等于根因分析。

连续追问

- 1. 如何判断是流量季节性还是系统故障？
- 2. 哪些监控应该预先建设？
- 3. 分数分布正常但 CTR 掉了说明什么？

10

如何评价一个推荐项目做得好不好？

★★★

项目拷打

30 秒回答

核心回答

项目评价至少需要四层证据：模型离线指标、召回/排序任务指标、线上业务指标、长期用户或生态指标；再加资源成本与稳定性。一个真正成功的项目要证明增益来自你的改动，并能稳定上线。

深入拆解

离线 AUC/GAUC/NDCG 只是筛选信号；召回看 Recall@K/HitRate；线上看 CTR、时长、CVR、GMV 等；长期看留存、活跃天数、满意度。

必须给 baseline、数据窗口、实验流量、置信区间和持续时间；只说“提升 2%”不够可信。

线上收益还要扣除计算、显存、延迟和维护成本，工业系统看 ROI。

公式 / 工程抓手

KEY

项目价值 $\approx$ Online Lift $\times$ Traffic Value - Serving/Engineering Cost

常见失分

只展示离线 AUC，或只展示一个没有实验设计信息的线上百分比。

连续追问

- 1. 提升来自模型还是数据？
- 2. 为什么离线涨线上不涨？
- 3. 如果成本翻倍但指标只涨 0.1% 是否上线？

召回、Embedding、双塔与负采样

召回是 2025-2026 推荐岗最容易拉开差距的模块。重点不只是双塔结构，而是负采样、In-batch bias、Hard Negative、ANN 与线上索引。

面试趋势

代表面经：快手追问召回训练目标与负样本；美团/阿里搜推公开面经出现 In-batch 偏差、InfoNCE 与负采样纠偏。

第 2 章 · 复习要点

11. UserCF 和 ItemCF 有什么区别？
12. ItemCF 有哪些经典改进？
13. 什么是矩阵分解？为什么能做推荐？
14. 双塔模型为什么适合大规模召回？
15. 双塔最大的缺点是什么？如何改？
16. 双塔通常怎么训练？InfoNCE 在做什么？
17. 为什么 In-batch Negative 很有效？
18. In-batch Negative 有什么严重问题？
19. Random Negative、Hard Negative、Semi-hard Negative 有什么区别？
20. 曝光未点击 Item 能不能直接当负样本？
21. InfoNCE 的 temperature 有什么作用？
22. 为什么热门 Item 不能无限作为负样本？
23. Embedding 维度应该怎么选？
24. 如何验证召回 Embedding 学得好不好？
25. ANN 为什么能解决大规模向量召回？HNSW、IVF、PQ 怎么选？

11 UserCF 和 ItemCF 有什么区别？

★★★

面经高频

CF / 相似度 / 共现

30 秒回答

核心回答

UserCF 通过相似用户迁移偏好，ItemCF 通过用户已交互 Item 找相似 Item。工业场景中 ItemCF 往往更稳定、可解释且容易离线预计算，但具体选择取决于用户/物料规模与变化速度。

深入拆解

UserCF 相似度容易受大用户群稀疏性影响；ItemCF 对“买了 A 的人也买 B”这类共现更自然。

两者都可加入时间衰减、活跃度惩罚和行为类型权重。

ItemCF 不是过时算法：它常作为高性价比召回路、冷启动前的稳定 baseline 和解释来源。

公式 / 工程抓手

KEY

$$\text{sim}(i,j)=|U_i \cap U_j| / \sqrt{|U_i| |U_j|}$$

常见失分

只背公式，不解释共现偏差和规模问题。

连续追问

1. 热门 Item 会不会和所有 Item 都相似？
2. 如何加入时间衰减？
3. ItemCF 与矩阵分解有什么关系？

12

ItemCF 有哪些经典改进？

★★★

面经高频

30 秒回答

核心回答

核心是修正“超级活跃用户”和“热门 Item”带来的虚假共现，并让近期、高价值行为贡献更大。

深入拆解

- 对用户 u 的共现贡献乘 $1/\log(1+|I_u|)$ ，降低超级活跃用户权重。
- 对行为加入时间衰减 $\exp(-\Delta t/\tau)$ ，对购买/收藏/深度观看赋更高权重。
- 相似度可做 shrinkage，避免低频 Item 仅凭一次共现得到极高相似度。

公式 / 工程抓手

KEY

$\text{sim}(i,j)=\sum_u w_u / \text{sqrt}(|U_i||U_j|)$ ， w_u 可组合活跃度惩罚与时间衰减。

常见失分

只做 cosine 共现，没有考虑热门偏置与低样本不稳定。

连续追问

- 1. 如何做离线更新与增量更新？
- 2. 低频 Item 相似度如何平滑？
- 3. 共现窗口怎么选？

13

什么是矩阵分解？为什么能做推荐？

★★★

面经高频

30 秒回答

核心回答

矩阵分解把稀疏用户-物品交互矩阵近似分解成低维用户向量与物品向量，假设偏好可由少量潜在因子解释。点积给出匹配分数。

深入拆解

- 显式反馈可最小化评分误差；隐式反馈常用加权 MF 或 BPR。
- MF 的价值是把“共现”泛化到潜在空间，但难直接利用复杂内容特征和上下文。
- 现代双塔可理解为把 user/item 向量从 lookup 参数升级为深度特征函数。

公式 / 工程抓手

KEY

$R \approx PQ$ ； $\text{score}(u,i)=p_u \cdot q_i$

常见失分

把未交互都当作明确负反馈。

连续追问

- 1. 隐式反馈如何训练 MF？
- 2. 为什么 BPR 比 MSE 更适合排序？
- 3. MF 与双塔的本质联系？

14

双塔模型为什么适合大规模召回？

★★★

面经高频

30 秒回答

核心回答

双塔把 User 和 Item 独立编码，线上只需算一次 User Embedding，并在预计算的 Item 向量库做 ANN 检索，因此能够把复杂深度模型与亿级候选搜索解耦。

深入拆解

Item Tower 可离线批量更新并建索引，在线计算复杂度从遍历全库变成近似最近邻。

点积/余弦具有可索引性；若最后用复杂 cross network，就失去 ANN 的优势。

工程上还要关注 embedding 归一化、向量版本、索引刷新和召回延迟。

公式 / 工程抓手

KEY

$u=f(x_u)$, $v=g(x_i)$, $score=u \cdot v$; TopK 由 ANN 完成。

常见失分

只说“两个塔分别编码”，不解释为什么能够 serving。

连续追问

1. 双塔为什么表达能力弱？
2. 怎么让双塔建模上下文？
3. MIPS 和 cosine 检索有什么区别？

15

双塔最大的缺点是什么？如何改？

★★★

面经高频

双塔瓶颈 / 多兴趣 / 蒸馏

30 秒回答

核心回答

最大问题是交互发生太晚：user 与 item 在最后点积前没有细粒度特征交叉，表达力受限。改进方向是在不破坏可索引性的前提下增强表示，或用多阶段架构把强交互留给排序。

深入拆解

可用更强序列编码器、多兴趣向量、知识蒸馏、teacher-student、query-aware user tower。

可用多向量检索：一个用户多个兴趣 embedding，提高召回覆盖。

若加入 cross encoder，通常放在粗排/精排而非大库召回。

公式 / 工程抓手

KEY

Efficiency Expressiveness 是双塔的核心 trade-off。

常见失分

为了“增强交互”直接把 user-item 拼接进一个网络，结果无法预计算 item。

连续追问

1. 多兴趣向量怎么索引？
2. 怎样蒸馏精排信息到召回？
3. 双塔召回分数如何校准？

16 双塔通常怎么训练？InfoNCE 在做什么？

★★★

面经高频

30 秒回答

核心回答

常用对比学习把正样本拉近、负样本推远。InfoNCE 将正 Item 与一组候选负 Item 共同归一化，相当于让模型在候选集合中识别真正例。

深入拆解

训练质量高度依赖负样本集合；网络加深不一定比负采样优化更有效。

相似度可用 dot-product/cosine，temperature 控制 softmax 尖锐程度。

大规模训练可结合 in-batch negatives、跨卡 negatives、memory queue。

公式 / 工程抓手

KEY

$$L = -\log \frac{\exp(s(u, i+)) / \tau}{\sum_j \exp(s(u, i_j) / \tau)}$$

常见失分

只讲公式，不讲负样本分布与线上候选分布的差异。

连续追问

1. 为什么需要 temperature？
2. 负例越多越好吗？
3. 跨卡 in-batch 会带来什么问题？

17 为什么 In-batch Negative 很有效？

★★★

面经高频

30 秒回答

核心回答

一个 batch 的其他正 Item 可以“免费”作为当前用户的负例，使 B 个正 pair 产生近似 B(B-1) 个对比关系，大幅提升负例数量与训练效率。

深入拆解

大 batch 会提升负例多样性，但也增加假负例概率和通信成本。

分布式训练常做 all-gather 扩大 negative pool；要考虑跨卡重复 Item。

如果 batch 是按时间/用户/类目构造，负样本分布可能不再随机，需要纠偏。

公式 / 工程抓手

KEY

对 u_i : $\text{positive}=i_i$; $\text{negatives}=\{i_j \mid j \neq i\}$

常见失分

把 batch 内所有其他 Item 都视为“确定不喜欢”。

连续追问

1. False Negative 怎么处理？
2. 为什么 batch size 变大会改变 loss？

3. 怎样做 logQ correction?

18 In-batch Negative 有什么严重问题?

★★★

面经高频

30 秒回答

核心回答

核心是 False Negative 和 sampling bias: 其他用户的正例可能也是当前用户潜在喜欢的 Item, 且高频 Item 更容易出现在 batch 中, 导致负样本分布偏离目标分布。

深入拆解

可用同 ID 去重、已交互 mask、类目/语义近邻软标签、多正例建模。

热门 Item 作为负例出现概率与其采样频率相关, 可使用 logQ/importance correction。

更稳妥的是 random + hard + debiased in-batch 混合。

公式 / 工程抓手

KEY

Observed negative $\neq$ true negative; 训练实际优化的是采样分布下的判别任务。

常见失分

只回答“假负例”, 不解释 popularity bias 与纠偏。

连续追问

1. logQ correction 为什么有效?
2. 多正例 loss 怎么写?
3. 怎样判断 hard negative 是不是假负例?

19 Random Negative、Hard Negative、Semi-hard Negative 有什么区别?

★★★

面经高频

30 秒回答

核心回答

随机负例覆盖面广但常太容易; Hard Negative 与正例相似, 能提供更大梯度但更可能是假负例; Semi-hard 在难度与可靠性之间折中。

深入拆解

Hard negative 可来自 ANN 近邻、上一版模型高分未点击、同类目相似 Item。

训练早期直接上极难负例可能不稳定, 可以 curriculum: 先随机后逐步加 hard。

在线曝光未点击是“行为负例”, 比随机库负例更接近线上分布, 但包含位置/曝光偏差。

公式 / 工程抓手

KEY

理想 negative: 有信息量, 但不是隐藏正例。

常见失分

把“越难越好”作为原则。

连续追问

1. 如何在线挖 hard negative?

2. 负采样比例怎么定？
3. 曝光未点击和随机负例如何混合？

20

曝光未点击 Item 能不能直接当负样本？

★★★

面经高频

30 秒回答

核心回答	不能直接等价。未点击可能因为用户没看到、位置太低、页面快速退出或展示方式差，因此它是带曝光机制偏差的弱负反馈。
------	---

深入拆解

- 可把“有效曝光未点击”定义得更严格，如停留到该位置、可见时长达到阈值。
- 对位置偏差可加 position feature、IPS 或随机干预估 propensity。
- 有明确负反馈（不感兴趣、踩、隐藏）时，应与普通未点击区分权重。

公式 / 工程抓手

KEY	$P(\text{click})=P(\text{examine} \text{position}) \cdot P(\text{preference} u,i,\text{context})$ 是常见点击建模拆分。
常见失分	把未点击全当 0 label，会让模型学习旧系统的曝光策略。

连续追问

1. 如何定义有效曝光？
2. 负反馈 label 权重怎么设计？
3. 位置偏差如何估 propensity？

21

InfoNCE 的 temperature 有什么作用？

★★★

面经高频

30 秒回答

核心回答	temperature τ 控制 softmax 分布的尖锐程度。 τ 越小，模型越强调高分 hard negatives，梯度更集中； τ 过小会放大 label noise、假负例和训练不稳定。
------	--

深入拆解

- τ 与 embedding norm、相似度函数共同决定 logit 尺度。若 embedding 未归一化，模型也可通过放大 norm 变相改变有效温度。
- 可将 τ 视作“区分难度旋钮”，需要结合 batch size、negative quality 调参。
- 监控 loss、logit 分布、recall、embedding norm，而不是只看训练 loss。

公式 / 工程抓手

KEY	$p(i u)=\exp(s_i/\tau)/\sum_j \exp(s_j/\tau)$
常见失分	只记“温度越小越尖锐”，没有谈梯度和噪声。

连续追问

1. 为什么 cosine 模型常需要归一化？
2. τ 是否可以学习？
3. 大 batch 是否应该调整 τ ？

22

为什么热门 Item 不能无限作为负样本？

★★★

面经高频

30 秒回答

核心回答

热门 Item 出现频率高，本身就更容易被采为 in-batch negative；若不纠偏，模型会被反复要求压低热门 Item 分数，形成过度反热门或概率失真。

深入拆解

可对采样概率 $q(i)$ 做 logQ correction / importance weighting。

也可用频率平滑 $q(i) \propto \text{freq}(i)^\alpha$ 、按 popularity bucket 采样。

目标不是消灭热门，而是让负例分布与部署目标一致。

公式 / 工程抓手

KEY

校正思路: $\text{logit}_i \leftarrow \text{logit}_i - \log q(i)$ （具体形式取决于目标与采样）。

常见失分

认为 popularity bias 一定要“打压热门”。很多热门本来就是真实高价值。

连续追问

1. 什么时候应该做 popularity debias？
2. logQ 校正会不会伤害 calibration？
3. 长尾采样比例怎么验证？

23

Embedding 维度应该怎么选？

★★

面经高频

30 秒回答

核心回答

维度不是按 ID 数量机械确定，而是容量、样本量、特征复杂度、内存和 serving 成本共同决定。高频特征可能值得更高维，长尾 ID 过高维只会浪费参数并过拟合。

深入拆解

内存约为 $\text{cardinality} \times \text{dim} \times \text{bytes}$ ；大表往往是推荐系统最大参数来源。

可采用 mixed-dimension embedding、hash、量化、低秩分解、频率感知分配。

离线用容量曲线，线上结合 latency/显存/网络开销做 Pareto 选择。

公式 / 工程抓手

KEY

$\text{Memory} \approx N \times d \times \text{precision_bytes}$

常见失分

“Item 越多, embedding 越大”是错误直觉。

连续追问

1. 如何做 mixed-dim?
2. embedding 表怎么做量化?
3. 维度过大有什么线上征兆?

24

如何验证召回 Embedding 学得好不好?

★★★

面经高频

30 秒回答

核心回答

最重要的是任务指标: Recall@K/HitRate/NDCG 和 ANN recall; 可视化只是辅助。还应检查向量范数、热门度相关性、类目/语义邻居、冷启动覆盖和线上增量。

深入拆解

区分“模型召回质量”和“ANN 索引近似误差”: 先 exact topK 再对比 ANN topK。
Case study 应看用户正例是否出现在近邻、相似 Item 是否合理, 同时警惕热门 Item 塌缩。
最终要看加入该召回回路后对后续排序与线上指标的边际增益。

公式 / 工程抓手

KEY

$$\text{Recall@K} = (\# \text{ test positives hit in TopK}) / (\# \text{ test positives})$$

常见失分

用 t-SNE 图“看起来聚类很好”当作主要证据。

连续追问

1. ANN recall 低怎么调?
2. embedding norm 与热门度高度相关怎么办?
3. 离线召回高线上无收益为什么?

25

ANN 为什么能解决大规模向量召回? HNSW、IVF、PQ 怎么选?

★★

论文追问

30 秒回答

核心回答

精确搜索对亿级向量代价过高, ANN 用少量召回精度换取数量级的延迟下降。HNSW 偏高召回低延迟但内存大; IVF 通过聚类缩小搜索桶; PQ 通过向量量化显著压缩内存。

深入拆解

选择要围绕 Recall-Latency-Memory 三角, 而不是只看一个指标。
HNSW 适合内存充足、在线高 recall; IVF/PQ 更容易在超大库做内存压缩。
还要考虑索引构建时间、增量更新、删除、冷热分层和分片。

公式 / 工程抓手

KEY	Exact: $O(Nd)$; ANN 通过图搜索/倒排聚类/量化减少访问向量数。
常见失分	把 HNSW、IVF、PQ 当成互斥算法; 工业上常组合 IVF+PQ 等。

连续追问

- 1. 索引刷新如何无损切换?
- 2. ANN recall 和业务 Recall@K 有何区别?
- 3. 动态 Item 库如何更新?

CTR 精排与特征交叉

这一章建立从 FM 到 DCNv2 的 feature interaction 主线，并把树模型、稀疏/连续特征和 offline-online gap 纳入统一理解。

面试趋势

代表面经：美团持续追问 DCN/DCNv2、PPNet；推荐岗仍常问 DeepFM、LGBM、特征选择与重要度。

第 3 章 · 复习要点

- 26. 为什么推荐排序需要特征交叉？
- 27. FM 为什么适合推荐系统？
- 28. Wide & Deep 的核心思想是什么？
- 29. DeepFM 与 Wide & Deep 最大区别是什么？
- 30. DeepFM 的 FM 二阶项怎么高效计算？
- 31. DCN 为什么比纯 MLP 更适合特征交叉？
- 32. DCNv2 改进在哪里？
- 33. XGBoost 和 LightGBM 有什么区别？
- 34. 稀疏特征和 Dense Feature 处理方式有什么不同？
- 35. 连续特征应该标准化还是分桶？
- 36. 标准化应该在 Embedding 拼接前还是拼接后？
- 37. 如何做特征选择与特征重要度分析？
- 38. 离线 AUC 提升，为什么线上可能没效果？

26

为什么推荐排序需要特征交叉？

★★★

面经高频

30 秒回答

核心回答	用户是否喜欢某个 Item 往往取决于 user、item、context 的组合，而不是单个特征独立贡献。特征交叉就是显式或隐式学习这些条件关系。
------	--

深入拆解

例如“上海+下雨+晚餐+火锅偏好”是联合模式；单独看每一项信号都弱。
树、FM、DCN、MLP、Attention 都可以视为不同形式的交互建模。
工业设计要考虑交互阶数、表达能力与计算效率，不能无限做全交叉。

公式 / 工程抓手

KEY	$score \neq f(user)+g(item)$ ，而更接近 $F(user,item,context,history)$ 。
-----	---

常见失分	把“交叉越多越好”当作结论。稀疏高阶交叉可能造成过拟合与巨大成本。
------	-----------------------------------

连续追问

- 1. FM 与 DCN 都做交叉，区别是什么？
- 2. MLP 为什么也能学交叉？
- 3. 如何解释某个交叉是否有效？

27

FM 为什么适合推荐系统？

★★★

面经高频

FM / 二阶交叉 / 稀疏

30 秒回答

核心回答	FM 用每个特征的低维隐向量内积代替独立二阶参数，因此在高维稀疏场景下，即使某个特征对组合很少共同出现，也能通过共享 latent factors 泛化。
------	---

深入拆解

普通二阶多项式需要 $O(n^2)$ 参数；FM 把 w_{ij} 分解为 $w_i \cdot w_j$ 。

计算可通过平方和技巧从 $O(n^2)$ 降为 $O(nd)$ 。

FM 主要建模二阶交互，复杂高阶关系需要 DeepFM/DCN 等。

公式 / 工程抓手

KEY	$y = w_0 + \sum_i w_i x_i + \sum_{i < j} \langle v_i, v_j \rangle x_i x_j$
常见失分	只说“FM 适合稀疏特征”，不解释为什么。

连续追问

- FM 如何降复杂度？
- field-aware FM 做了什么？
- FM 与矩阵分解的关系？

28

Wide & Deep 的核心思想是什么？

★★★

面经高频

30 秒回答

核心回答	Wide 部分擅长记忆人工交叉的确定模式，Deep 部分通过 embedding+MLP 泛化到未见组合，二者联合兼顾 Memorization 与 Generalization。
------	--

深入拆解

Wide 可放线性与人工 cross features；Deep 使用稀疏 embedding 和 dense features。

不足是 Wide 仍较依赖人工特征工程；这也是 DeepFM 的重要动机。

线上要注意两部分输入处理和联合校准。

公式 / 工程抓手

KEY	$y = \sigma(y_{wide} + y_{deep})$
常见失分	把 Wide 等同于 FM；原始 Wide 通常是广义线性模型与人工交叉。

连续追问

- DeepFM 如何改进？
- Wide 真的还需要吗？
- 哪些业务规则适合放 Wide？

29

DeepFM 与 Wide & Deep 最大区别是什么？

★★★

面经高频

30 秒回答

核心回答

DeepFM 用 FM 自动学习低阶交叉，并与 Deep 部分共享 embedding 输入，减少人工 cross feature 设计；Wide&Deep 的 Wide 更依赖人工构造。

深入拆解

FM 部分负责一阶+二阶，Deep 部分负责高阶非线性。

共享 embedding 降低冗余，但也意味着两个分支的表示相互影响。

DeepFM 是经典 baseline，不代表在所有业务都优于 DCNv2/Transformer。

公式 / 工程抓手

KEY

DeepFM = FM(low-order) + DNN(high-order), shared embeddings.

常见失分

回答“DeepFM 多了 FM”但说不清为什么这能减少人工交叉。

连续追问

1. DeepFM 的二阶项怎么算？
2. FM 分支与 DNN 分支梯度会冲突吗？
3. 为什么后来又有 DCN？

30

DeepFM 的 FM 二阶项怎么高效计算？

★★★

面经高频

FM 公式 /
复杂度推导

30 秒回答

核心回答

利用“平方和减平方的和”把所有 pair 内积的计算从显式 $O(n^2d)$ 化为 $O(nd)$ ，这是 FM 的经典技巧。

深入拆解

对每个 latent 维度 f ，先求 $\sum_i v_{\{i,f\}}x_i$ ，再平方；减去 $\sum_i v_{\{i,f\}}^2 x_i$ ，最后乘 $1/2$ 。

这个推导体现了为什么 FM 能在稀疏高维特征上高效训练。

面试时能现场推导通常比只背公式更加分。

公式 / 工程抓手

KEY

$$0.5 \cdot \sum_f [(\sum_i v_{if} x_i)^2 - \sum_i (v_{if} x_i)^2]$$

常见失分

把第二项写错，或忘记 $1/2$ 导致 pair 被重复计数。

连续追问

1. 推导复杂度为什么从 n^2d 变 nd ？
2. x_i 多数为 0 时还能怎么优化？

31 DCN 为什么比纯 MLP 更适合特征交叉？

★★★

面经高频

30 秒回答

核心回答

DCN 用 Cross Network 显式构造有界阶数的交叉，使网络对 feature interaction 有结构先验；MLP 虽理论上能学交叉，但需要更多参数和数据去“自己发现”。

深入拆解

Cross layer 通过 x_0 与当前 x_1 外积式交互逐层提高多项式阶数。

DCN 常与 DNN 并联：Cross 负责显式交叉，Deep 负责一般非线性。

原始 DCN 的表达能力受 rank-1 形式限制，引出 DCNv2。

公式 / 工程抓手

KEY

$$x_{l+1} = x_0(x_1 \cdot w_l) + b_l + x_1$$

常见失分

把 DCN 理解成简单残差 MLP。关键在 x_0 与 x_1 的显式乘性交互。

连续追问

1. 第 L 层最高能产生几阶交叉？
2. DCN 与 FM 的差异？
3. 为什么 DCN 原版表达力有限？

32 DCNv2 改进在哪里？

★★★

面经高频

30 秒回答

核心回答

DCNv2 将原 DCN 的向量式 cross 提升为更有表达力的矩阵交叉，并提供低秩与 mixture 版本，在 web-scale ranking 中兼顾表达能力与成本。

深入拆解

原 DCN 的 cross 近似 rank-1；DCNv2 使用 W_l 对交互进行更丰富线性变换。

低秩分解 $W=UV$ 控制参数与计算；MoE 可进一步让不同 expert 学不同交互子空间。

它适合作为“强但可控”的 feature crossing building block。

公式 / 工程抓手

KEY

$$x_{l+1} = x_0 \odot (W_l x_1 + b_l) + x_1 \quad (\text{常见表述之一})$$

常见失分

只说“矩阵更大所以更强”，不解释低秩与效率权衡。

连续追问

1. Low-rank rank 怎么选？
2. Stacked 与 Parallel DCNv2 有何区别？
3. 为什么 web-scale 更偏好可控交叉？

33

XGBoost 和 LightGBM 有什么区别？

★★

面经高频

30 秒回答

核心回答

两者都是梯度提升树，但 LightGBM 更强调大规模训练效率：histogram、leaf-wise、生长策略、GOSS/EFB 等；XGBoost 的正则与二阶优化框架更经典。推荐场景中树模型仍是强 baseline 和特征分析工具。

深入拆解

Leaf-wise 更容易快速降低 loss，但小数据可能过拟合，需要 max_depth/num_leaves 约束。

Histogram 将连续值离散成桶，降低内存和计算。

推荐中常用于粗排、融合特征、teacher 或解释 feature importance。

公式 / 工程抓手

KEY

不要只回答 “LightGBM 更快”；说明为什么快、代价是什么。

常见失分

把 GOSS/EFB 当作所有 LightGBM 配置都会启用的必然步骤。

连续追问

1. Leaf-wise 为什么易过拟合？
2. 树模型怎么处理类别特征？
3. DNN 与 GBDT 如何融合？

34

稀疏特征和 Dense Feature 处理方式有什么不同？

★★

面经高频

30 秒回答

核心回答

ID/类别特征是离散语义，不应把数值大小当作连续关系，通常通过 embedding；连续特征则可标准化、log、分桶或直接输入网络。

深入拆解

高基数类别要处理 OOV、hash 冲突和长尾。

连续特征的尺度会影响优化，长尾数值常用 loglp。

同一连续特征可同时保留 raw normalized 和 bucket embedding，让模型兼顾平滑趋势与非线性阈值。

公式 / 工程抓手

KEY

ID 12345 与 12346 的数值相近没有语义，因此不能按连续距离处理。

常见失分

把所有整数型字段都当 categorical，或所有数值都直接喂网络。

连续追问

1. 价格特征为什么常做 log？
2. 时间特征如何周期编码？

3. OOV ID 怎么处理？

35

连续特征应该标准化还是分桶？

★★

面经高频

30 秒回答

核心回答

两者解决不同问题。标准化保留连续顺序并改善优化；分桶让模型容易学习阈值和非线性，还能缓解极端值。工业上常同时使用。

深入拆解

如果关系近似单调平滑，标准化直接输入很自然；如果存在明显区间效应，bucket embedding 更强。

分桶策略可等宽、等频、业务阈值或可学习边界。

必须避免离线/在线分桶边界不一致。

公式 / 工程抓手

KEY

 $x_{\text{norm}} = (x - \mu) / \sigma$; $\text{bucket}(x) \rightarrow \text{embedding}$

常见失分

把分桶当成“丢失信息所以一定不好”，或标准化当成所有模型必需。

连续追问

1. 树模型是否需要标准化？
2. 分桶数量怎么选？
3. 动态分布下边界怎么维护？

36

标准化应该在 Embedding 拼接前还是拼接后？

★★

面经高频

30 秒回答

核心回答

数据预处理的标准化主要针对 dense features，通常在拼接前独立处理；Embedding 是可学习向量，不用普通 z-score。拼接后可使用 LayerNorm/BatchNorm，但那属于模型内部归一化。

深入拆解

区分 feature scaling 与 representation normalization。

若连续特征直接和 embedding 拼接，量纲过大可能主导前层梯度。

向量召回里还可能对 user/item embedding 做 L2 normalization，这是第三种概念。

公式 / 工程抓手

KEY

 $\text{Feature scaling} \neq \text{LayerNorm} \neq \text{L2-normalized retrieval embedding}$

常见失分

把几种“归一化”混在一起。

连续追问

1. 什么时候用 BatchNorm，什么时候 LayerNorm？

2. L2 normalization 对点积检索有什么影响？

37 如何做特征选择与特征重要度分析？

★★

面经高频

30 秒回答

核心回答	可靠的特征选择要结合统计、模型和实验。互信息/相关性用于初筛，树模型 importance 或 gate/attention 提供线索，真正判断增量价值要做 feature ablation 和线上验证。
------	--

深入拆解

- 注意 permutation importance 和 gain importance 都可能受特征相关性影响。
- 高基数 ID 的 importance 不一定可直接与 dense feature 比。
- 删特征不仅看效果，还要看特征计算成本、freshness 和故障风险。

公式 / 工程抓手

KEY	$\Delta_feature = Metric(All) - Metric(All - feature)$
常见失分	把 attention weight 当成严格因果解释。

连续追问

- 两个强相关特征如何判断谁重要？
- 特征删掉 AUC 不变但线上涨了为什么？
- 如何评估实时特征价值？

38 离线 AUC 提升，为什么线上可能没效果？

★★★

面经高频

30 秒回答

核心回答	离线和线上之间存在样本分布、候选分布、特征新鲜度、概率校准、策略覆盖和时延等多重断层。AUC 涨只说明离线 pair ordering 改善，不保证最终业务 utility 改善。
------	--

深入拆解

- 提升可能集中在不重要的 easy samples 或非头部位置。
- 旧日志由旧策略曝光产生，存在 counterfactual 缺失；新模型上线会改变分布。
- 线上重排/规则可能把模型增益覆盖掉；额外 latency 也会抵消收益。

公式 / 工程抓手

KEY	Offline metric 是必要但非充分条件。
常见失分	用一句“线上线下分布不一致”结束，没有定位框架。

连续追问

- 怎样设计更贴近线上分布的离线评估？

2. 模型 calibration 与 AUC 有什么关系?
 3. 如何做 shadow test?
-

用户行为序列与长兴趣建模

从 DIN 的 target-aware interest 出发，经过 DIEN 的兴趣演化、SIM 的检索式长历史，到 Transformer/HSTU 的序列 scaling。

面试趋势

代表面经：DIN/DIEN/SIM 是稳定高频；近年进一步追问长序列效率、LayerNorm、HSTU。

第 4 章 · 复习要点

39. DIN 为什么出现？它解决了什么问题？
40. DIN 的 Attention 具体怎么计算？
41. DIN 和 Self-Attention 有什么区别？
42. DIN 有什么缺点？
43. DIEN 相比 DIN 改进在哪里？
44. DIEN 为什么需要 Auxiliary Loss？
45. GRU、AGRU、AUGRU 有什么区别？
46. 为什么用户行为序列需要去重、去噪和辅助信息？
47. 超长用户序列为什么不能直接上普通 Transformer？
48. SIM 如何处理超长行为序列？
49. Transformer 为什么适合用户行为序列？
50. 为什么推荐 Transformer 常用 LayerNorm 而不是 BatchNorm？

39

DIN 为什么出现？它解决了什么问题？

★★★

面经高频

30 秒回答

核心回答

传统 Embedding+Pooling 把用户历史压成一个固定向量，无论候选是什么都一样。DIN 用 target-aware local activation，让“与当前候选相关的历史兴趣”被更强激活。

深入拆解

同一用户面对篮球鞋和相机，应使用不同历史证据。

DIN 的用户表示是 candidate-conditioned，因此表达更强，但也增加每个候选的计算。

它是“兴趣匹配”思想的代表，不是简单把 Transformer attention 搬进 CTR。

公式 / 工程抓手

KEY

$$v_u(\text{target}) = \sum_i a(e_i, e_{\text{target}}) \cdot e_i$$

常见失分

把 DIN 说成普通 self-attention。

连续追问

1. DIN attention 和 Transformer attention 有何区别？
2. 长历史时 DIN 为什么贵？
3. DIN 是否能建模兴趣演化？

40

DIN 的 Attention 具体怎么计算？

★★★

面经高频

30 秒回答

核心回答

对每个历史行为 embedding 与 target embedding 构造匹配特征，经小 MLP 得到 activation score，再对历史向量加权汇聚。原始 DIN 的 local activation 不要求标准 softmax 归一化。

深入拆解

常见匹配输入包含 e_i 、 e_t 、 $e_i - e_t$ 、 $e_i \odot e_t$ 。

不做 softmax 时，权重总量也可以表达兴趣强度；这与概率注意力不同。

实际工程中可替换匹配网络，但核心仍是 target-conditioned pooling。

公式 / 工程抓手

KEY

$$a_i = \text{MLP}([e_i, e_t, e_i - e_t, e_i \odot e_t]); v = \sum_i a_i e_i$$

常见失分

强行套 QK / $\sqrt{d}$ 的 Transformer 公式。

连续追问

1. 为什么不一定 softmax？
2. 候选很多时怎么缓存？
3. 如何加入时间特征？

41

DIN 和 Self-Attention 有什么区别？

★★★

面经高频

30 秒回答

核心回答

DIN 主要回答“历史中的哪些行为与当前 target 相关”；Self-Attention 主要回答“历史行为之间如何相互影响”。前者是 target-to-history，后者是 history-to-history。

深入拆解

DIN 每个候选会产生不同用户表示。

Self-Attention 能建模序列内部上下文与多头关系，但计算常为 $O(L^2)$ 。

现代模型常组合：先 self-attention 编码历史，再 target attention 读取。

公式 / 工程抓手

KEY

$$\text{DIN: } a(\text{history}, \text{target}); \text{ SelfAttn: } a(\text{history}_i, \text{history}_j)$$

常见失分

只说“DIN 是单头，Transformer 是多头”。这不是本质区别。

连续追问

1. Cross-Attention 与 DIN 更像吗？
2. 什么时候只用 DIN 更划算？

42

DIN 有什么缺点？

★★★

面经高频

DIN limitations

30 秒回答

核心回答

DIN 没有显式建模兴趣随时间演化，长序列下逐候选 attention 成本高，且历史之间的依赖较弱。DIEN、SIM、Transformer/HSTU 分别从演化、搜索长历史和序列建模角度补足。

深入拆解

target-aware 很强，但 candidate-dependent 计算难充分缓存。

如果用户历史数千甚至数万，直接遍历所有行为不现实。

历史噪声、重复行为、时间间隔也需要额外机制。

公式 / 工程抓手

KEY

核心瓶颈：sequence length $\times$ candidate count 的在线成本。

常见失分

只回答“模型老了”。

连续追问

1. DIEN 如何改进？
2. SIM 如何降低长序列成本？
3. DIN 为什么仍然常用？

43

DIEN 相比 DIN 改进在哪里？

★★★

面经高频

30 秒回答

核心回答

DIEN 把“行为”与“潜在兴趣”区分开：先用 GRU 提取随时间变化的兴趣状态，并用辅助损失监督；再用与 target 相关的兴趣演化层建模哪些兴趣影响当前目标。

深入拆解

Interest Extractor 通过序列网络形成 h_t ；auxiliary loss 让 h_t 能预测下一行为。

Interest Evolution 通过 AIGRU/AGRU/AUGRU 把 target relevance 注入状态更新。

因此 DIEN 不只是“DIN+GRU”，而是显式建模 interest evolution。

公式 / 工程抓手

KEY

$h_t = \text{GRU}(e_t, h_{t-1})$ ；auxiliary next-behavior supervision

常见失分

只说“DIEN 多了 GRU”，没有解释 auxiliary loss 与 evolution。

连续追问

1. 辅助 loss 的正负样本怎么构造？
2. AUGRU 为什么优于 AGRU？

44

DIEN 为什么需要 Auxiliary Loss?

★★★

面经高频

30 秒回答

核心回答

如果只有最终点击 label, GRU 中间状态未必真的表示“兴趣”。辅助损失让每个 h_t 对下一步真实行为高分、对负样本低分,使中间状态获得密集监督。

深入拆解

它缓解长链路训练中监督过稀的问题。

负样本仍要考虑采样质量,否则辅助任务也会被噪声污染。

辅助任务权重过大可能让模型过度优化 next-item,而不是最终 CTR。

公式 / 工程抓手

KEY

$$L_{total} = L_{ctr} + \alpha \sum_t L_{aux}(h_t, e_{\{t+1\}}, \hat{e}_t)$$

常见失分

把辅助 loss 解释成普通正则项。

连续追问

1. α 怎么选?
2. 如果序列不是严格时间行为怎么办?
3. 能否用 contrastive auxiliary objective?

45

GRU、AGRU、AUGRU 有什么区别?

★★

论文追问

30 秒回答

核心回答

GRU 由 update gate 自主决定状态更新; AGRU 用 attention score 直接替换 update gate; AUGRU 将 attention 与原 update gate 相乘,更细粒度地保留 GRU 自身状态控制。

深入拆解

AGRU 的整个 hidden vector 使用同一个 attention 标量更新。

AUGRU 先有原始 u_t , 再乘 target relevance a_t , 兼顾序列动态与目标相关性。

面试更关注“attention 如何进入 recurrent update”,而不是背缩写。

公式 / 工程抓手

KEY

$$\text{AUGRU: } \tilde{u}_t = a_t \cdot u_t$$

常见失分

把 AIGRU、AGRU、AUGRU 混为一谈。

连续追问

1. AIGRU 是怎么做的?
2. 为什么标量 attention 可能限制表达?

46

为什么用户行为序列需要去重、去噪和辅助信息？

★★★

面经高频

30 秒回答

核心回答

原始日志不是纯兴趣序列。重复刷新、误触、异常播放、系统重试会让行为次数与真实偏好不成比例；时间、行为类型、停留深度等辅助信息有助于判断信号质量。

深入拆解

去重可按 item+时间窗口，去噪可基于有效观看阈值、异常设备/机器人规则。

同一 Item 多次交互不一定全部删除：重复购买或多次深度观看可能是真强兴趣，应区分原因。

加入 action type、dwell time、time gap、position 等能帮助模型解释行为语义。

公式 / 工程抓手

KEY

 $\text{Raw events} \rightarrow \text{validated events} \rightarrow \text{semantic behavior sequence}$

常见失分

粗暴把重复全部删除，可能损失真实强兴趣。

连续追问

1. 重复次数应该作为 feature 还是保留多 token？
2. 实时序列和离线序列如何一致？

47

超长用户序列为什么不能直接上普通 Transformer？

★★★

面经高频

30 秒回答

核心回答

标准 self-attention 的时间和显存复杂度随序列长度平方增长。L 从 1k 到 10k 时 attention matrix 放大 100 倍，工业高 QPS 精排难以承受。

深入拆解

除了 $O(L^2)$ ，逐候选在线编码也会造成巨额延迟。

可以通过截断、稀疏 attention、检索式历史选择、分层建模、缓存或专用架构降低成本。

长历史还包含大量不相关行为，先搜索相关片段常比全量建模更有效。

公式 / 工程抓手

KEY

 $\text{Self-Attention complexity} \approx O(L \cdot d)$

常见失分

只说“Transformer 慢”，不量化长度平方带来的增长。

连续追问

1. FlashAttention 是否解决了 $O(L^2)$ ？
2. 为什么 SIM 先 search？
3. HSTU 解决的是什么？

48

SIM 如何处理超长行为序列？

★★★

面经高频

30 秒回答

核心回答

SIM 的核心是两阶段：GSU 在超长历史中快速搜索与 target 相关的子序列，ESU 再对小集合做精细建模。“先检索相关历史，再建模”把计算从全量序列转移到有价值片段。

深入拆解

GSU 可用类别/embedding 近似相关性；ESU 使用更强的 attention/sequence model。

搜索阶段本身也要避免错过跨类目兴趣，因此可多路或学习检索。

这是长期历史建模中很通用的 retrieve-then-model 思路。

公式 / 工程抓手

KEY

History $10^4 \sim 10^5 \rightarrow$ GSU TopM $\rightarrow$ ESU exact modeling

常见失分

只说“SIM 能建模 5 万序列”，不解释两阶段原理。

连续追问

1. GSU 召回错了怎么办？
2. 如何设置 M？
3. SIM 与 RAG 有什么结构类比？

49

Transformer 为什么适合用户行为序列？

★★

面经高频

30 秒回答

核心回答

它具备并行训练、长距离依赖和多头关系建模能力，能把购买、点击、观看等行为作为序列 token 学习上上下文。但推荐数据与语言不同，必须处理高基数 ID、时间间隔、异构动作和在线延迟。

深入拆解

Position encoding 在推荐里常需加入真实 time gap，而不仅是绝对位置。

多头可捕获不同兴趣，但不能自动等同于“多个可解释兴趣”。

序列很长时需要专门的效率设计。

公式 / 工程抓手

KEY

$\text{Attention}(Q,K,V) = \text{softmax}(QK^T / \sqrt{d})V$

常见失分

把语言 Transformer 原样复制到推荐，不谈数据结构差异。

连续追问

1. 时间间隔怎么编码？
2. 是否需要 causal mask？

3. 用户序列中 CLS token 有何作用？

50

为什么推荐 Transformer 常用 LayerNorm 而不是 BatchNorm？

★★★

面经高频

30 秒回答

核心回答	LayerNorm 在单样本 hidden 维度归一化，不依赖 batch 组成，更适合变长序列、分布波动和线上单请求推理；BatchNorm 依赖 batch 统计，train/eval 统计差异更明显。
------	---

深入拆解

- 推荐 batch 往往跨用户、序列长度和活跃度差异大，batch statistics 不稳定。
- LayerNorm 与 padding mask 的处理相对自然。
- 但 DNN dense tower 中 BatchNorm 仍可能有效，不应绝对化。

公式 / 工程抓手

KEY	$LN(x)=(x-\mu_{features})/\sqrt{\sigma^2_{features}+\epsilon}$
常见失分	回答 “Transformer 标配 LayerNorm” 而没有解释 batch 依赖。

连续追问

- Pre-LN vs Post-LN？
- RMSNorm 有什么区别？

CHAPTER 05

多任务、多目标与 CVR 建模

掌握 Shared Bottom→MMoE→PLE 的负迁移主线，以及 CTR/CVR 条件链、ESMM 的 entire-space 思想。

面试趋势

代表面经：美团高频出现 MMoE/PLE/跷跷板、ESMM/ESCM2、连续时长目标。

第 5 章 · 复习要点

51. 为什么推荐系统需要 Multi-Task Learning?
52. Shared Bottom 有什么问题?
53. MMoE 怎么解决任务冲突?
54. MMoE 和普通 MoE 最大区别是什么?
55. 什么叫多任务“跷跷板现象”?
56. PLE 相比 MMoE 改进在哪里?
57. 多任务 Loss 怎么加权?
58. 一个任务收敛了，另一个还没收敛怎么办?
59. CTR、CVR 为什么不能直接独立训练?
60. ESMM 的核心原理是什么?
61. ESMM 有什么不足? 后续模型为什么继续改?
62. 连续目标（如观看时长）应该怎么建模?

51

为什么推荐系统需要 Multi-Task Learning?

★★★

面经高频

30 秒回答

核心回答

用户反馈天然多目标且有层级关系：曝光→点击→深度行为→转化。MTL 共享表示可以利用辅助信号、减少重复计算，并同时优化多个业务目标。

深入拆解

共享能缓解稀疏任务数据不足，但任务关系不一致会产生 negative transfer。

多任务不等于多 loss；结构、梯度、样本空间和最终 score fusion 都重要。

目标之间可能是相关、竞争、条件概率或因果链，结构应匹配关系。

公式 / 工程抓手

KEY

$$L=\sum_t \lambda_t L_t$$

常见失分

默认所有任务都共享同一底座并等权相加。

连续追问

1. Shared Bottom 为什么会失败?
2. 任务 label 频率差异怎么处理?
3. 线上分数如何融合?

52

Shared Bottom 有什么问题？

★★★

面经高频

30 秒回答

核心回答	所有任务强制共享同一表示，当任务相关性较弱或目标冲突时，梯度会互相干扰，导致负迁移和“一个任务涨、另一个任务掉”的跷跷板。
------	---

深入拆解

- 可以观察 task gradients cosine similarity、各任务 loss 曲线、共享层激活。
- 简单增加 tower 不能解决共享层冲突。
- MMoE/PLE 通过更灵活的共享结构缓解问题。

公式 / 工程抓手

KEY	若 $L_a \cdot L_b < 0$ ，说明局部梯度方向冲突。
常见失分	只回答“任务不同所以不好”，缺少机制。

连续追问

- 1. 哪些任务适合 Shared Bottom？
- 2. 梯度冲突如何直接处理？

53

MMoE 怎么解决任务冲突？

★★★

面经高频

30 秒回答

核心回答	MMoE 用多个共享 experts 产生不同表示，再为每个任务训练独立 gate，允许不同任务选择不同 expert mixture，从硬共享变为条件化软共享。
------	---

深入拆解

- Experts 仍是共享的，task-specificity 主要由 gate 体现。
- 若所有 gate 最终选择相同 experts，退化成强共享；若 expert collapse，需要负载/结构诊断。
- MMoE 对任务相关性不确定的场景比 Shared Bottom 更鲁棒。

公式 / 工程抓手

KEY	$h_t = \sum_k g_{\{t,k\}}(x) E_k(x)$ ， $g_t = \text{softmax}(W_t x)$
常见失分	把每个 expert 当成对应一个任务。MMoE experts 本身共享。

连续追问

- 1. expert 数怎么选？
- 2. gate 输入用什么？
- 3. MMoE 的缺点是什么？

54

MMoE 和普通 MoE 最大区别是什么？

★★★

面经高频

30 秒回答

核心回答	普通 MoE 的重点是为样本路由到 experts；MMoE 的关键是“每个任务有自己的 gate”，同一个样本在不同任务下可以得到不同 expert mixture。
------	--

深入拆解

- MMoE 的目标是建模 task relationship，而非主要追求稀疏计算。
- 现代 Sparse-MoE 关注 top-k routing 与负载均衡，概念上要区分。
- 推荐 MTL 中 experts 可以是 MLP、Cross、序列模块等。

公式 / 工程抓手

KEY	Task t has gate g_t ; same experts E_k are shared.
常见失分	把 MMoE 与 LLM Sparse MoE 的计算稀疏化目标混在一起。

连续追问

- 1. 能否做 task-specific experts?
- 2. PLE 为什么要进一步拆 experts?

55

什么叫多任务“跷跷板现象”？

★★★

面经高频

30 秒回答

核心回答	优化一个任务常导致另一个任务下降，例如 CTR 上升但时长下降。这可能来自梯度冲突、目标权重、数据空间差异或任务本身存在竞争。PLE 论文将该现象作为重要动机之一。
------	--

深入拆解

- 分析时先判断是模型共享导致，还是业务目标真实冲突。
- 可看 per-task gradient、gate/expert usage、不同用户分层的 trade-off。
- 解决方法包括 PLE、梯度平衡、动态 loss weighting、约束优化。

公式 / 工程抓手

KEY	Seesaw $\neq$ 一定是模型 bug；可能反映真实 Pareto trade-off。
常见失分	只背名词，不会诊断。

连续追问

- 1. 如何判断是 negative transfer 还是业务真实冲突？
- 2. 上线决策怎么处理 Pareto frontier？

56

PLE 相比 MMoE 改进在哪里？

★★★

面经高频

30 秒回答

核心回答	PLE 显式区分 shared experts 与 task-specific experts，并逐层进行信息抽取，让任务既能共享共性又能保留独有表示，减少无关信息被强制共享。
------	--

深入拆解

- 每个 CGC 层包含共享 expert、任务 expert 和对应 gates。
- 多层 progressive extraction 让高层逐渐形成更纯的任务表示。
- 代价是结构和参数更复杂，需要调 expert 数与层数。

公式 / 工程抓手

KEY	$\text{Task representation} = \text{gated}(\text{task-specific experts} + \text{shared experts})$
常见失分	把 PLE 说成 “MMoE 多一层”。核心是显式的共享/独享 expert 分解。

连续追问

- 1. 共享 expert 数量怎么选？
- 2. 为什么 task-specific expert 能缓解负迁移？

57

多任务 Loss 怎么加权？

★★★

面经高频

30 秒回答

核心回答	最简单是手工 λ_t ，但不同任务 loss 尺度、稀疏度和业务价值不同。更系统的方法包括 uncertainty weighting、GradNorm、动态权重、梯度投影或基于线上 utility 的约束优化。
------	--

深入拆解

- 先把 “优化稳定性” 和 “业务权重” 区分：loss scale 归一化不等于业务价值权重。
- 稀疏任务可能需要更高采样或辅助任务，而不是简单放大 λ 。
- 动态方法也可能震荡，要监控 per-task gradient norm。

公式 / 工程抓手

KEY	$L = \sum_t \lambda_t L_t$ ； λ 可以静态或动态。
常见失分	直接按业务重要性把 loss 权重设成 100:1，可能造成训练数值失衡。

连续追问

- 1. GradNorm 的直觉？
- 2. 两个 loss 数量级差 1000 倍怎么办？

58

一个任务收敛了，另一个还没收敛怎么办？

★★★

面经高频

30 秒回答

核心回答	不能只让训练继续“拖着走”。可通过任务权重衰减、分任务学习率、冻结部分 head、stop-gradient 或梯度平衡，让未收敛任务继续学习，同时保护已稳定任务。
------	--

深入拆解

- 先确认“收敛”是训练 loss 还是验证指标 plateau。
- 共享层仍在更新时，即使冻结 head，已收敛任务表现也可能变化。
- 如果任务学习速度长期差异巨大，可能需要重新设计结构/样本。

公式 / 工程抓手

KEY	核心是控制 shared representation 的后续梯度影响。
常见失分	只回答“早停”。多任务不能对单个任务简单全局早停。

连续追问

- 1. 冻结 head 为什么仍可能掉？
- 2. 如何做 per-task LR？

59

CTR、CVR 为什么不能直接独立训练？

★★★

面经高频

30 秒回答

核心回答	CVR label 通常只有点击后才可观察，训练样本来自 clicked subset，而推理面对全部曝光，产生 Sample Selection Bias；转化又极稀疏，存在 Data Sparsity。
------	---

深入拆解

- 直接在点击样本训练 CVR 学到的是 selection-conditioned distribution。
- 点击本身受旧系统排序影响，因此 clicked subset 不是随机样本。
- ESMM 利用曝光→点击→转化的概率关系在 entire space 学习。

公式 / 工程抓手

KEY	$CVR=P(conv click)$ ；训练域 clicked $\neq$ 推理域 all impressions
常见失分	把未点击全部当“未转化”直接训练 CVR，会改变条件概率定义。

连续追问

- 1. ESMM 怎么避免 SSB？
- 2. CVR 与 CTCVR 有什么关系？

60

ESMM 的核心原理是什么？

★★★

面经高频

30 秒回答

核心回答	利用概率链 $P(\text{click} \cap \text{conversion}) = P(\text{click}) \cdot P(\text{conversion} \text{click})$ ，同时在全曝光空间监督 CTR 与 CTCVR，通过共享表示间接学习 CVR，缓解点击样本选择偏差和转化稀疏。
------	--

深入拆解

- CTR head 预测 pCTR；CVR head 预测 pCVR；两者乘积作为 pCTCVR 与曝光空间标签训练。
- CVR head 不直接用“仅点击样本”的 CVR loss，这是 entire-space 思想关键。
- 共享 embedding/底座让大量点击信号帮助转化任务。

公式 / 工程抓手

KEY	$pCTCVR = pCTR \times pCVR$
常见失分	把 ESMM 误写成 “CTR loss + CVR loss”。

连续追问

- 1. 为什么 pCVR 仍可能有估计偏差？
- 2. 如果有加购等中间行为怎么办？

61

ESMM 有什么不足？后续模型为什么继续改？

★★★

论文追问

30 秒回答

核心回答	ESMM 解决了经典 SSB/稀疏问题，但 pCVR 是通过乘法间接约束，可能出现估计偏差、概率空间与条件链假设问题；后续 ESCM、ESM、ESMC 等尝试从反事实、行为分解或参数约束继续改进。
------	--

深入拆解

- 面试不要求背所有后续模型，但要知道 ESMM 并非无偏真理。
- 中间行为（加购、收藏、进店）可以提供更密集监督，但行为图的概率链必须定义严谨。
- 高级回答应讨论 counterfactual missingness 与 exposure policy。

公式 / 工程抓手

KEY	模型结构必须与真实行为概率图相匹配。
常见失分	把论文改进模型名称堆一串，但说不清 ESMM 的根本限制。

连续追问

- 1. ESCM 的反事实思想是什么？
- 2. 中间行为如何建图？

62

连续目标（如观看时长）应该怎么建模？

★★

面经高频

30 秒回答

核心回答

观看时长常 heavy-tail、零膨胀且受内容长度强烈影响，不宜直接裸 MSE。可 log-transform、分桶分类、序数回归、分布建模或预测 normalized watch ratio，再结合校准。

深入拆解

MSE 会被少数超长样本主导；Huber/loglp 更稳健。

短视频完播率天然偏短内容；可同时建模 duration 或分时长桶校准。

如果最终排序关心期望时长，可预测分布后计算 $E[T|x]$ 。

公式 / 工程抓手

KEY

$y = \log(1 + \text{watch_time})$ 或建模 $p(\text{bucket_k} | x)$ 后求期望。

常见失分

直接回归 raw seconds 且忽略视频长度偏差。

连续追问

- 时长和 CTR 如何多任务？
- 完播率为什么有 duration bias？

Loss、排序指标与在线实验

面试官通过 AUC/GAUC、NDCG、Time Split 与 A/B，判断你是否真正理解“模型指标”和“业务决策”的差距。

面试趋势

代表面经：快手直接追问 AUC 在不平衡场景的定义与适用性；各大厂项目拷打都要求解释线上实验。

第 6 章 · 复习要点

- 63. 为什么 CTR 常用 Binary Cross Entropy?
- 64. Pointwise、Pairwise、Listwise 有什么区别?
- 65. AUC 的本质是什么?
- 66. 正负样本严重不平衡时 AUC 为什么仍较稳定?
- 67. AUC 有什么缺点? 为什么推荐常看 GAUC?
- 68. GAUC 具体怎么算? 权重怎么选?
- 69. Precision、Recall、F1 在推荐中怎么用?
- 70. NDCG 为什么适合推荐排序?
- 71. 离线评价为什么必须重视 Time Split?
- 72. A/B Test 如何分桶?
- 73. 为什么 A/B 前最好做 A/A?
- 74. CTR 显著上涨但停留时长下降, 怎么办?
- 75. 线上提升 0.1% 到底算不算有效?

63

为什么 CTR 常用 Binary Cross Entropy?

★★★

面经高频

30 秒回答

核心回答	点击是 Bernoulli 事件, BCE 等价于对 Bernoulli likelihood 做最大似然; 其梯度与 sigmoid 结合稳定, 并直接学习概率。
------	--

深入拆解

- BCE 关注概率校准, AUC 关注排序, 两者目标并不完全相同。
- 采样后如果正负先验改变, 输出概率需做校准/采样修正。
- 类别极不平衡时可以加权, 但权重会改变概率语义。

公式 / 工程抓手

KEY	$L=-y \log p-(1-y) \log (1-p)$
常见失分	说“因为二分类所以 BCE”但不理解概率含义。

连续追问

- 1. BCE 与 focal loss 区别?
- 2. 为什么 AUC 涨而 logloss 变差?

64

Pointwise、Pairwise、Listwise 有什么区别？

★★★

面经高频

30 秒回答

核心回答

Pointwise 独立预测每个 Item 的 label; Pairwise 直接约束正 Item 排在负 Item 前; Listwise 把整个列表顺序作为优化对象。选择取决于 label 形式、线上目标和训练效率。

深入拆解

CTR 预测天然 pointwise; BPR 是典型 pairwise; LambdaRank/ListNet 属于 listwise/近似 listwise 思路。

Pairwise 更贴近排序，但采样 pair 的方式会显著影响目标。

Listwise 更直接但通常计算与工程复杂度更高。

公式 / 工程抓手

KEY

$$\text{BPR: } L = -\log \sigma(\hat{s}^+ - \hat{s}^-)$$

常见失分

认为 ranking 就一定要 listwise。工业精排大量仍用 pointwise 多任务。

连续追问

1. 为什么 pointwise 也能得到好排序？
2. Pairwise 如何做负采样？

65

AUC 的本质是什么？

★★★

面经高频

30 秒回答

核心回答

AUC

可以解释为随机取一个正样本和一个负样本，模型给正样本更高分的概率，因此它与阈值无关，直接反映 pairwise ranking 能力。

深入拆解

AUC 对单调变换不敏感，因此不能评价概率校准。

在极端不平衡数据上仍较稳定，但业务可能更关心头部 precision/recall。

推荐里跨用户 pair 可能没有意义，引出 GAUC。

公式 / 工程抓手

KEY

$$\text{AUC} = P(\text{score}(x^+) > \text{score}(x^-))$$

常见失分

只会画 ROC，不会给概率解释。

连续追问

1. AUC=0.5/1 分别代表什么？
2. 为什么 calibration 不影响 AUC？

66

正负样本严重不平衡时 AUC 为什么仍较稳定？

★★★

面经高频

30 秒回答

核心回答	ROC 的 TPR 在正类内部归一化，FPR 在负类内部归一化，因此类数量比例变化不会像 Accuracy 一样直接主导指标。
------	---

深入拆解

“稳定”不等于“永远合适”：如果业务关注极低 FPR 区间，整体 AUC 可能掩盖头部性能。

PR-AUC 对稀少正类更敏感。

采样方式会影响 score distribution，但理论 AUC 与先验比例关系较弱。

公式 / 工程抓手

KEY	$TPR=TP/(TP+FN)$ ； $FPR=FP/(FP+TN)$
常见失分	说“AUC 完全不受样本不平衡影响”。实际训练和采样仍会改变模型。

连续追问

1. 什么时候用 PR-AUC？
2. 为什么线上 CTR 很低但 AUC 仍高？

67

AUC 有什么缺点？为什么推荐常看 GAUC？

★★★

面经高频

30 秒回答

核心回答	全局 AUC 会把不同用户之间本不需要比较的分数也纳入 pair。例如一个用户整体分数偏高、另一个偏低，跨用户 pair 会影响指标；真实推荐只需在同一用户候选内排序。
------	--

深入拆解

GAUC 对每个 user/group 单独算 AUC 再加权，更贴近 per-user ranking。

加权可按曝光数、点击数或业务价值，但口径要固定。

极少正负样本的 user 无法计算 AUC，需要过滤或平滑。

公式 / 工程抓手

KEY	$GAUC=\sum_u w_u AUC_u / \sum_u w_u$
常见失分	把 GAUC 简单说成“更准确的 AUC”，不解释 group 的意义。

连续追问

1. group 应该按 user 还是 request？
2. 如何处理全正/全负 group？

68

GAUC 具体怎么算？权重怎么选？

★★★

面经高频

GAUC calculation

30 秒回答

核心回答	先按 user/request 分组计算组内 AUC，再按曝光量、有效样本量或业务权重做加权平均。权重决定指标更关注大用户还是平均用户。
------	--

深入拆解

按曝光加权更接近总体流量价值；等权更强调每个用户公平贡献。

group 太小时 AUC 方差高，可以设置最小正负样本数。

面试中应明确分组粒度与权重，不要只写公式。

公式 / 工程抓手

KEY	$GAUC = \sum_g n_g \cdot AUC_g / \sum_g n_g$ （常见实现）
常见失分	忽略无法计算 AUC 的 group。

连续追问

- GAUC 与 request-level AUC 有何区别？
- 权重变化会怎样影响模型选择？

69

Precision、Recall、F1 在推荐中怎么用？

★★★

面经高频

30 秒回答

核心回答	Precision 衡量推荐出来的有多少是相关，Recall 衡量真实相关中找回多少。召回阶段更重 Recall@K，最终 TopK 也可看 Precision/NDCG；F1 适合需要单值平衡的场景。
------	---

深入拆解

推荐离线往往只有观测到的少量正反馈，未交互不一定是不相关，因此绝对 Precision/Recall 受评估协议影响。

K 的选择必须与线上列表长度/候选规模一致。

召回模型常使用 HitRate@K/Recall@K 而不是分类阈值指标。

公式 / 工程抓手

KEY	$Precision = TP / (TP + FP)$ ； $Recall = TP / (TP + FN)$
常见失分	把分类任务的阈值 Precision 直接套进 TopK 推荐，不说明 K。

连续追问

- HitRate@K 与 Recall@K 区别？
- 多正例用户怎么计算？

70

NDCG 为什么适合推荐排序？

★★★

面经高频

30 秒回答

核心回答	NDCG 同时考虑相关性等级和位置折扣，高相关结果排得越靠前贡献越大，并用理想排序归一化到可比较区间。
------	--

深入拆解

- 如果 label 只有 0/1，NDCG 仍体现正例位置；若有多级反馈，可以给不同 gain。
- log 折扣表达“越靠后价值越低”，但折扣形式本身是人为设计。
- NDCG 是离线排序指标，不代表用户真实停留价值。

公式 / 工程抓手

KEY	$DCG@K = \sum_i (2^{\{rel_i\}} - 1) / \log_2(i+1)$ ； $NDCG = DCG / IDC$
常见失分	把 IDC 忘掉，或认为 NDCG 可以跨不同 K 直接比较。

连续追问

- MRR 与 NDCG 区别？
- gain 应该如何映射点击/购买？

71

离线评价为什么必须重视 Time Split？

★★★

面经高频

30 秒回答

核心回答	推荐问题的真实部署是“用过去预测未来”。随机切分可能把未来交互、未来热门趋势甚至未来 Item 信息泄漏到训练，导致离线结果虚高。
------	---

深入拆解

- 应按时间构造 train/validation/test，并保证特征也只使用当时可见数据。
- 新 Item/新用户在未来测试集出现正是 cold-start 的真实部分，不应全部过滤。
- 时间切分还可以检测 distribution shift 和模型 freshness。

公式 / 工程抓手

KEY	Train: $t < t_0$; Test: $t \geq t_0$
常见失分	只切 label 时间，却用全量历史统计特征，仍然泄漏。

连续追问

- 负样本如何按时间采？
- Time split 会不会让测试更难？

72

A/B Test 如何分桶？

★★★

面经高频

30 秒回答

核心回答

通常对稳定实体 (user_id/device_id) 与 experiment_id 做 hash，映射到固定 bucket，保证同一用户持续处于同组，同时尽量随机、均匀且实验间可正交。

深入拆解

分桶实体取决于干预范围：跨设备用户体验需尽量 user-level。

需要检查 SRM（样本比例失配）和跨组污染。

多个实验同时运行时要有层/域设计，避免冲突。

公式 / 工程抓手

KEY

$$\text{bucket} = \text{hash}(\text{unit_id}, \text{exp_id}) \bmod N$$

常见失分

用每次 request 随机分组，会让同一用户在实验/对照间跳动。

连续追问

1. 什么是实验污染？
2. 用户登录前后如何一致？
3. SRM 怎么发现？

73

为什么 A/B 前最好做 A/A？

★★★

面经高频

30 秒回答

核心回答

A/A 让两组运行完全相同策略，用来验证分桶、埋点、指标管道和统计检验是否会“凭空制造差异”。若 A/A 经常显著，说明实验基础设施有问题。

深入拆解

可检查均值、方差、SRM、关键人群分布。

A/A 不是每个实验都必须长期跑，但新平台/新指标/大改埋点时非常有价值。

应预先定义显著性阈值，不要看到差异后再改规则。

公式 / 工程抓手

KEY

$$H_0: \text{Treatment 与 Control 同分布}$$

常见失分

认为 A/A 的结果必须每个指标完全相等；随机波动仍然存在。

连续追问

1. A/A 显著怎么办？
2. 如何减少指标方差？

74

CTR 显著上涨但停留时长下降，怎么办？

★★★

面经高频

30 秒回答

核心回答	这说明模型可能提升了“点击诱惑”但损伤消费质量。应查看有效点击、完播、session 时长、负反馈、次留等指标，并重新审视多目标价值函数，而不是只因 CTR 显著就上线。
------	---

深入拆解

- 先判断时长下降来自视频长度结构变化还是用户更快退出。
- 可按内容长度、人群、流量位切片，定位 trade-off 来源。
- 如果北极星更接近时长/留存，CTR 只能作为诊断指标。

公式 / 工程抓手

KEY	上线决策遵循 North Star + Guardrails，而非“谁显著谁赢”。
常见失分	只做一个加权分数，未解释权重依据。

连续追问

- 1. 如何判断 clickbait？
- 2. 如果时长短期掉但次留涨呢？

75

线上提升 0.1% 到底算不算有效？

★★★

系统设计

30 秒回答

核心回答	看 effect size、置信区间、样本量、持续时间、业务规模与成本。大流量产品 0.1% 可能价值巨大，小流量实验则可能只是噪声。统计显著不等于业务显著。
------	---

深入拆解

- 报告应包含绝对值、相对 lift、CI/p-value、实验天数、流量比例。
- 要考虑多重比较、novelty effect、周周期和节假日。
- 上线还要比较 serving cost: 0.1% 收益换 2 倍 GPU 可能不划算。

公式 / 工程抓手

KEY	$Decision \approx BusinessValue(lift) - IncrementalCost - Risk$
常见失分	只说 $p < 0.05$ 就上线。

连续追问

- 1. 最小可检测效应 MDE 是什么？
- 2. 实验要跑多久？

偏差、数据与在线工程

推荐日志是旧策略干预后的观测数据。去偏、采样、特征一致性、在线训练与延迟优化是从算法走向工程的关键。

面试趋势

系统题高频: Position/Exposure Bias、Train-Serving Skew、分布漂移、P99 latency、Embedding 大表。

第 7 章 · 复习要点

- 76. 推荐数据有哪些常见 Bias?
- 77. Position Bias 怎么解决?
- 78. IPS 是什么? 有什么问题?
- 79. 怎么解决类别不平衡?
- 80. 为什么训练样本不能随便随机负采样?
- 81. 什么是 Train-Serving Skew?
- 82. 在线训练和离线训练有什么区别?
- 83. 如何解决数据分布漂移?
- 84. 推荐模型线上延迟过高怎么优化?
- 85. Embedding 表太大怎么办?

76

推荐数据有哪些常见 Bias?

★★★

面经高频

30 秒回答

核心回答	核心包括 Exposure Bias、Position Bias、Popularity Bias、Selection Bias、Survivorship Bias 等。推荐日志不是自然世界随机采样，而是旧策略干预后的结果。
------	---

深入拆解

- 曝光偏差：没展示的 Item
- 没有反馈；位置偏差：顶部更易点击；热门偏差：热门获得更多数据形成反馈环；选择偏差：如 CVR 只在 clicked subset 可观察。
- 训练模型如果忽略这些机制，会复制旧策略甚至放大偏差。
- 处理方式包括随机干预、propensity/IPS、反事实学习、采样修正和多样性探索。

公式 / 工程抓手

KEY	$\text{Observed log} = \text{User preference} \times \text{Logging policy} \times \text{UI/exposure mechanism}$
常见失分	把 bias 只理解成“数据不均衡”。

连续追问

- 1. 哪个 bias 对 CTR 影响最大?
- 2. 去偏会不会降低短期指标?

77

Position Bias 怎么解决？

★★★

面经高频

30 秒回答

核心回答

先承认点击是“看见 × 喜欢”的结果。可将 position 作为特征、用随机交换位置估 propensity、训练 click model，或用 IPS 对不同曝光概率重权。

深入拆解

只把 position 当 feature 可以拟合偏差，但未必得到无偏 relevance。

随机干预是估计 position propensity 的强证据，但要控制用户体验风险。

实际常做 propensity clipping，防止低概率位置权重爆炸。

公式 / 工程抓手

KEY

$$P(\text{click}) = P(\text{examine} | \text{position}) \cdot P(\text{relevant} | u, i)$$

常见失分

把位置 feature 加进模型就声称“去偏”。那可能只是更好地利用偏差。

连续追问

1. 线上不能大规模随机怎么办？
2. position bias 与 presentation bias 区别？

78

IPS 是什么？有什么问题？

★★★

面经高频

30 秒回答

核心回答

IPS 用样本被日志策略选择/曝光的概率倒数做权重，让低曝光概率样本代表更多未观测样本，从而估计目标策略下的无偏期望。主要问题是 propensity 很小时方差爆炸。

深入拆解

常用 clipping、SNIPS、自归一化或 Doubly Robust 降方差。

propensity 本身估计错误会引入新偏差。

IPS 适用于 counterfactual evaluation/learning，但需要明确 logging policy。

公式 / 工程抓手

KEY

$$L_{\text{IPS}} = \sum_i o_i / p_i \cdot L_i$$

常见失分

只背“ $1/p$ 加权”，不讲高方差与 positivity assumption。

连续追问

1. 为什么需要 overlap/positivity？
2. DR 为什么更稳？

79

怎么解决类别不平衡？

★★★

面经高频

30 秒回答

核心回答

可从采样、loss、任务建模三层处理：负样本下采样、class weight/focal loss、hard negative、多任务辅助等。但任何采样都可能改变输出概率，需要 calibration 或先验修正。

深入拆解

不要把“负样本多”本身当问题；排序任务可能正需要大量负例。

Focal loss 更关注难样本，但噪声 label 也可能被放大。

线上概率若用于价值融合，校准尤其重要。

公式 / 工程抓手

KEY

训练先验 π_{train} 与线上 π_{online} 不一致时，需要 prior correction。

常见失分

下采样后直接把 sigmoid 输出当真实 CTR。

连续追问

- 如何做概率校准？
- focal loss 何时不适合？

80

为什么训练样本不能随便随机负采样？

★★★

面经高频

30 秒回答

核心回答

负采样定义了模型看到的“对比世界”，会改变决策边界。全库随机负例通常太容易且与线上曝光分布差异大；只用曝光未点击又会继承旧策略 bias。

深入拆解

理想采样应兼顾代表性与信息量：random、exposed、hard、fresh 等混合。

采样概率最好可记录，以便 importance correction。

评估时也要区分 sampled metrics 与 full-catalog metrics。

公式 / 工程抓手

KEY

模型优化的是 $P_{\text{sample}}(y|x)$ ，不是自动等于 $P_{\text{online}}(y|x)$ 。

常见失分

只考虑正负比例，不考虑负例来源。

连续追问

- 召回与精排的负采样是否应相同？
- 如何做动态 hard negative mining？

81

什么是 Train-Serving Skew?

★★★

面经高频

30 秒回答

核心回答	训练特征/处理逻辑与线上实际特征不一致，导致离线模型学到部署时不存在的信息或不同分布，是“离线好、线上崩”的典型根因。
------	---

深入拆解

例：训练使用当天最终统计，线上只能看到请求时累计值；训练用未来窗口统计是更严重 leakage。

解决靠统一 feature definition、Feature Store、point-in-time join、线上回放测试。

监控线上/离线特征分布、缺失率、版本号。

公式 / 工程抓手

KEY	$x_{\text{offline}}(t)$ 必须等价于请求时可见的 $x_{\text{online}}(t)$ 。
常见失分	把 skew 仅理解为模型版本不一致。

连续追问

- 1. Point-in-time correctness 是什么？
- 2. 如何做线上特征回放？

82

在线训练和离线训练有什么区别？

★★★

面经高频

30 秒回答

核心回答	离线训练按小时/天批量使用时间窗数据，稳定、易回溯；在线/增量训练持续吸收最新反馈，freshness 更高，但受到 label delay、噪声、稳定性和灾难性遗忘挑战。
------	--

深入拆解

推荐兴趣变化快，纯日更可能滞后；可采用小时级增量而非真正逐样本更新。

在线训练要设计 checkpoint、回滚、异常流量保护和 delayed label join。

新数据权重过高会忘记长期兴趣，需要 replay 或混合窗口。

公式 / 工程抓手

KEY	Batch window vs streaming updates 是连续谱，不是二元。
常见失分	认为“在线训练一定更先进更好”。

连续追问

- 1. 转化 label 延迟怎么处理？
- 2. 线上训练出错怎么回滚？

83

如何解决数据分布漂移？

★★★ 面经高频

30 秒回答

核心回答	先区分 covariate shift $P(X)$ 变化、label/conditional shift $P(Y X)$ 变化以及业务策略引起的 feedback loop，再选择时间衰减、滑窗训练、增量学习、重采样或触发重训。
------	--

深入拆解

- 监控 PSI/KL、特征均值分位、score/label calibration、分人群指标。
- 热点/节日是可预期 drift，可加入时间上下文；产品改版可能是结构性 drift。
- 模型 freshness 与稳定性要权衡，不能遇到 drift 就极端缩短窗口。

公式 / 工程抓手

KEY	监控 $P_t(X)$, $P_t(Y X)$, Metric_t by segment
常见失分	只监控 overall AUC，细分人群已漂移却发现不了。

连续追问

- 1. 如何选择训练窗口长度？
- 2. 概念漂移和数据异常怎么区分？

84

推荐模型线上延迟过高怎么优化？

★★★ 面经高频

30 秒回答

核心回答	按“少算、复用、近似、压缩、硬件友好”逐层优化：减少候选、缓存/预计算 Item 侧、蒸馏、量化、batching、算子融合、GPU-friendly architecture，并用 profile 找真正瓶颈。
------	--

深入拆解

- 先拆 latency：特征读取、网络、召回、模型、后处理；不要盲目换 GPU。
- 高 QPS 下 P99 比平均值更重要，缓存击穿和长尾请求常决定体验。
- 模型结构要考虑 MFU 与访存；RankMixer 这类工作正针对传统 ranking 的硬件低利用。

公式 / 工程抓手

KEY	$Latency_budget = feature + retrieval + inference + rerank + network$
常见失分	只优化 FLOPs，忽略 embedding lookup 和网络 I/O。

连续追问

- 1. 为什么 batch inference 会增大等待？
- 2. P50/P99 怎么权衡？

85

Embedding 表太大怎么办？

★★

面经高频

30 秒回答

核心回答	大规模推荐的参数主要可能来自 embedding table。常见方案有 hash、频率裁剪、mixed-dimension、量化、低秩分解、冷热分层、分布式参数服务器和淘汰机制。
------	--

深入拆解

- 十亿级 ID × 128 dim × FP32 已是数百 GB。
- 高频 ID 值得独立表示，低频 ID 可共享 hash bucket 或更低维。
- 量化不仅省存储，也减少网络带宽，但需关注召回相似度精度。

公式 / 工程抓手

KEY	Memory=N×d×bytes; 10^9×128×4≈512GB
常见失分	只说“分布式存储”，没有减少参数本身。

连续追问

- 1. hash collision 怎么处理？
- 2. INT8 embedding 对召回影响怎么评估？

冷启动、多样性、长尾与探索

推荐系统不仅要“猜用户会点什么”，还要主动获取信息、维护新内容与供给生态，并控制反馈回路。

面试趋势

场景题高频：新用户/新物料、保量、信息茧房、多样性与热门反馈环。

第 8 章 · 复习要点

- 86. 什么是 User Cold Start? 怎么做?
- 87. Item Cold Start 怎么做?
- 88. User Cold Start 和 Item Cold Start 最大区别是什么?
- 89. 如何解决热门 Item 吞噬长尾 Item?
- 90. 怎么评价推荐多样性?
- 91. 如何在准确率和多样性之间 trade-off?
- 92. 如何解决“信息茧房”?
- 93. 为什么新内容需要“保量”? 如何保量?

86

什么是 User Cold Start? 怎么做?

★★★

面经高频

30 秒回答

核心回答	新用户没有历史行为，协同与长序列模型无法发挥。策略应从内容/上下文/热门先验出发，通过 onboarding、地域时间、设备场景和探索尽快获得高信息量反馈。
------	--

深入拆解

- 目标不是一开始“猜得最准”，而是用较低风险推荐同时学习用户偏好。
- 可设计多样化探索列表，避免只推荐全站热门导致用户画像迟迟不分化。
- 注册属性要谨慎使用，不应压过实时行为。

公式 / 工程抓手

KEY	$Cold\text{-}start\ objective \approx utility + information\ gain$
常见失分	一上来要求用户选择几十个兴趣标签，增加 onboarding 摩擦。

连续追问

- 1. 探索比例怎么定?
- 2. 匿名用户如何跨 session 建模?

87

Item Cold Start 怎么做?

★★★

面经高频

30 秒回答

核心回答	新 Item 没交互数据，但通常有文本、图片、类目、作者等内容特征。可通过内容 encoder 生成初始 embedding，并给探索曝光收集早期反馈，再逐渐切换到协同信号。
------	---

深入拆解

内容 embedding 可与协同 embedding 对齐/蒸馏，避免新旧 Item 表示空间割裂。

探索要控制质量和安全，按作者信誉/内容质量做先验。

新 Item 的真实问题是“缺 interaction evidence”，不是完全没有信息。

公式 / 工程抓手

KEY	<code>e_new=f(text,image,metadata); then online feedback updates prior</code>
常见失分	只用热门作者先验，可能抑制真正的新作者。

连续追问

- 1. 如何做 content-collaborative alignment?
- 2. 新 Item 保量多久?

88

User Cold Start 和 Item Cold Start 最大区别是什么?

★★★

面经高频

30 秒回答

核心回答	User cold start 缺的是“偏好证据”，需要探索用户；Item cold start 缺的是“交互证据”，需要理解内容并探索物料。二者的探索对象和信息获取路径不同。
------	--

深入拆解

新用户可通过少量高区分度 Item 快速识别兴趣；新 Item 则要在合适人群中测试接受度。

两者都要避免 feedback loop，但 prior 来源不同。

系统设计中常需要单独的 cold-start traffic policy。

公式 / 工程抓手

KEY	User: learn preference; Item: learn quality/audience fit
常见失分	把两者都回答成“推荐热门”。

连续追问

- 1. 双冷启动（新用户+新 Item）怎么办?

89

如何解决热门 Item 吞噬长尾 Item?

★★★

面经高频

30 秒回答

核心回答	热门内容因更多曝光获得更多点击数据，模型更有信心，又继续给更多曝光，形成 popularity feedback loop。可以做采样修正、分数去热门、多样性配额和探索，但要保持真实用户价值。
------	--

深入拆解

- 区分“受欢迎”与“被系统放大”：不能为了公平机械打压用户真正喜欢的热门内容。
- 可看 exposure concentration、catalog coverage、Gini/Herfindahl 等。
- 长尾保护可放在召回配额或重排约束，不一定强行修改精排概率。

公式 / 工程抓手

KEY	Exposure→Feedback→TrainingData→Score→MoreExposure
常见失分	只谈长尾曝光，不验证用户满意度是否下降。

连续追问

- 1. 如何衡量曝光集中度?
- 2. 长尾 Item 是否都值得曝光?

90

怎么评价推荐多样性?

★★★

面经高频

30 秒回答

核心回答	多样性包括列表内差异、全局覆盖、作者/类目分布、新颖性等。可用 Intra-List Similarity、Coverage、Entropy、Gini 等，并与准确率/满意度联合评估。
------	--

深入拆解

- ILS 越低通常越多样，但“随机垃圾”也很多样，所以不能单独优化。
- Coverage 衡量系统是否只反复推荐少数 Item；用户级 entropy 可衡量兴趣跨度。
- 多样性最好按用户兴趣宽度自适应。

公式 / 工程抓手

KEY	$Coverage = \bigcup_u Rec(u) / I $; $ILS \approx avg\ pairwise\ similarity$
常见失分	把多样性指标当最终目标。

连续追问

- 1. 多样性和新颖性区别?
- 2. 用户只喜欢单一类目怎么办?

91

如何在准确率和多样性之间 trade-off?

★★★

面经高频

30 秒回答

核心回答	常用 rerank 在 relevance 基础上对与已选集合过于相似的候选施加惩罚，如 MMR；也可设作者/类目 quota、DPP 或约束优化。权重需要 A/B 验证。
------	--

深入拆解

- 列表构建是 sequential decision: 每选一个 Item，后续候选的边际价值改变。
- 不同用户兴趣熵不同， λ 可个性化。
- 业务上还要处理广告、运营位、新内容等硬约束。

公式 / 工程抓手

KEY	$MMR(i) = \lambda \cdot Rel(i) - (1 - \lambda) \cdot \max_{j \in S} Sim(i, j)$
常见失分	直接随机打散精排 TopK。

连续追问

- 1. 如何学习 λ ？
- 2. DPP 的直觉是什么？

92

如何解决“信息茧房”？

★★★

面经高频

30 秒回答

核心回答	本质是 exploitation 反馈越来越强化已知兴趣，需要受控 exploration，引入跨兴趣但仍有潜在相关性的内容，并监控长期满意度，而不是随机塞无关内容。
------	---

深入拆解

- 可用 ϵ -greedy、UCB、Thompson Sampling、novelty/diversity constraints。
- 探索应考虑不确定性：模型越不确定的候选越有信息价值。
- 长期衡量用户兴趣覆盖、内容供给与留存，而非只看当次点击。

公式 / 工程抓手

KEY	Policy balances immediate reward + exploration value
常见失分	把“信息茧房”简化成“多推荐不同类目”。

连续追问

- 1. Bandit 与普通 A/B 有何不同？
- 2. 探索流量会不会伤害短期 CTR？

93

为什么新内容需要“保量”？如何保量？

★★

系统设计

30 秒回答

核心回答

纯监督模型对无历史 Item 往往打低分，导致无曝光、无数据、继续低分。保量是人为提供最小探索机会，让系统获得早期质量证据。

深入拆解

可以建立 fresh recall channel、按内容质量先验分配探索 quota、按不确定性逐步放量。

保量不是无条件保送，应有快速淘汰与安全阈值。

观察早期反馈时要修正探索位置和人群差异。

公式 / 工程抓手

KEY

No exposure→No data→Low confidence→No exposure

常见失分

给所有新内容固定大流量，可能造成体验和安全风险。

连续追问

1. 如何定义新内容毕业条件？
2. 探索数据能否直接混入训练？

2026 前沿：HSTU、RankMixer 与生成式推荐

这一章不是为了追热点，而是理解推荐系统正在发生的两条 Scaling 路线：序列/生成式建模与硬件友好的 Ranking Scaling。

面试趋势

前沿证据：ICML 2024 Generative Recommenders/HSTU；2025 RankMixer；近期搜推面试已开始直接追问。

第 9 章 · 复习要点

94. HSTU 为什么可能比普通 Transformer 更适合推荐?
95. 什么是 Generative Recommendation?
96. 生成式召回和双塔召回有什么区别?
97. 为什么生成式推荐常需要 Semantic ID / RQ-VAE?
98. RankMixer 想解决什么问题?
99. LLM 怎么进入推荐系统?
100. 如果重新设计下一代推荐系统, 你选传统 Pipeline 还是生成式推荐?

94

HSTU 为什么可能比普通 Transformer 更适合推荐?

★★★

前沿高频

30 秒回答

核心回答	推荐序列具有高基数 ID、非平稳流式行为、长序列和异构动作，语言 Transformer 的设计并非天然最优。HSTU 面向生成式推荐重新设计序列 transduction，强调在长推荐序列上的效率与 scaling。
------	---

深入拆解

2024 Generative Recommenders 工作把推荐重写为 sequential transduction，并提出 HSTU。

面试重点不是背具体 layer，而是理解“推荐数据分布与语言不同，架构应围绕行为序列与硬件约束重新设计”。

它代表一条路线：从传统 feature-based DLRM 转向 sequence-centric foundation recommender。

公式 / 工程抓手

KEY	核心问题: quality 是否能随 compute/data/model size 稳定 scaling，同时满足线上成本。
常见失分	把 HSTU 说成“更快的 Transformer”就结束。

连续追问

1. 它如何处理长序列?
2. 为什么 generative framing 有 scaling 优势?

95

什么是 Generative Recommendation?

★★★

前沿高频

30 秒回答

核心回答	传统推荐通常对候选逐个打分；生成式推荐把用户行为历史当序列，直接建模下一个 Item/语义 ID 的条件概率，把检索与用户建模向 next-token prediction 范式靠拢。
------	--

深入拆解

目标可写为 $P(i_{t+1} | i_1 \cdots i_t, context)$ 。

优势是统一序列建模和可能的 scaling law；挑战是超大 Item vocabulary、解码效率、约束和冷启动。

“生成式”不意味着必须输出自然语言。

公式 / 工程抓手

KEY	$P(\text{next_item} \mid \text{behavior_history}, \text{context})$
常见失分	认为生成式推荐就是“用 LLM 写推荐理由”。

连续追问

1. 如何生成十亿 Item?
2. 线上 beam search 会不会太慢?

96

生成式召回和双塔召回有什么区别?

★★★

前沿高频

30 秒回答

核心回答	双塔将用户映射为向量并通过 ANN 在 Item embedding 库检索；生成式召回直接预测 Item token/semantic code 序列。前者成熟高效，后者更强调统一序列概率建模和表达能力。
------	---

深入拆解

- 双塔天然支持高效 MIPS；生成式需要解决 vocabulary/semantic ID 和解码约束。
- 双塔的 item 表示可独立预计算；生成式模型的候选概率依赖整个历史上下文。
- 现实架构很可能长期混合：传统 retrieval 做高吞吐，生成模型做用户表征或补充召回。

公式 / 工程抓手

KEY	TwoTower: $\text{TopK } u \cdot v_i$; Generative: $\text{argmax}_i P(i \mid \text{history})$
常见失分	回答“生成式一定取代双塔”。忽略 QPS、延迟和运维成熟度。

连续追问

1. 二者如何蒸馏?
2. 生成式召回如何保证合法 Item?

97

为什么生成式推荐常需要 Semantic ID / RQ-VAE?

★★★

论文追问

30 秒回答

核心回答	如果每个 Item 直接作为独立 token，十亿级词表难以训练和解码。Semantic ID 用多个离散 code 表示一个 Item，把大词表分解成短 code sequence，并让相似内容共享部分语义代码。
------	---

深入拆解

- RQ-VAE/残差量化可将连续内容/协同 embedding 映射到多级 codebooks。
- 语义 ID 的好坏决定生成空间结构：code collision、codebook collapse、Item 唯一性都要处理。
- 新 Item 可通过 encoder 获得 code，有利于冷启动。

公式 / 工程抓手

KEY	$\text{Item} \rightarrow [c_1, c_2, \dots, c_m], c_k \in \text{Codebook_k}$
常见失分	把 semantic ID 只当压缩技巧；它同时改变生成任务的结构先验。

连续追问

- 1. 如何保证 Item 唯一映射？
- 2. codebook size 与 sequence length 怎么权衡？

98

RankMixer 想解决什么问题？

★★★

前沿高频

30 秒回答

核心回答	传统 ranking 模型包含大量 CPU 时代遗留的手工交叉模块，GPU MFU 低，模型难随 compute 扩大。RankMixer 用硬件友好的 token mixing 统一 feature interaction，并探索 Dense/Sparse-MoE scaling。
------	--

深入拆解

- 其 2025 工作报告把 MFU 从约 4.5% 提升到约 45%，并在近似相同推理延迟下把参数规模扩大约 100 倍。
- 关键启示：推荐 scaling 不只是“堆参数”，要让算子结构适配 GPU，并控制 serving ROI。
- 它与 HSTU 代表不同侧重点：一个偏 ranking scaling，一个偏 sequence/generative recommender。

公式 / 工程抓手

KEY	Scaling target = model quality $\uparrow$ while latency/QPS/cost remain acceptable
常见失分	把 RankMixer 当作又一个 feature crossing layer，而不谈硬件效率和 scaling。

连续追问

- 1. MFU 为什么重要？
- 2. 为什么 self-attention 在 ranking token 数上未必划算？

99

LLM 怎么进入推荐系统？

★★★

前沿高频

30 秒回答

核心回答	LLM 可以作为语义能力组件，而不是简单替换整条推荐链路：Query/Intent 理解、内容 embedding、用户历史摘要、特征生成、rerank、解释，以及生成式推荐都是可行入口。
------	--

深入拆解

- 最有价值的场景通常是传统 ID 模型缺乏语义或跨域泛化的地方。
- 直接用大模型在线逐候选打分成本高，可用蒸馏、小模型、缓存或离线 embedding。
- 推荐效果必须以线上用户价值验证，不能因为“用了 LLM”就默认更强。

公式 / 工程抓手

KEY	LLM value = semantic generalization + reasoning; constraint = latency/cost/data freshness
常见失分	把 LLM 当成万能精排器，忽略成本与实时性。

连续追问

- 1. LLM 用户画像会不会过度压缩行为？
- 2. 如何把 LLM teacher 蒸馏给双塔？

100

如果重新设计下一代推荐系统，你选传统 Pipeline 还是生成式推荐？

★★★

系统设计

30 秒回答

核心回答	成熟答案不是二选一，而是按阶段和约束组合：高吞吐召回继续使用协同/双塔/ANN，长序列用户建模吸收 HSTU/Transformer，排序向 RankMixer/大规模硬件友好模型演进，语义与冷启动使用 LLM/多模态，最终通过多目标与约束重排完成。
------	---

深入拆解

首先给业务约束：QPS、SLA、Item 规模、反馈密度、成本；然后决定哪些阶段值得生成化。

生成式方法的优势是统一建模与 scaling 潜力；传统 pipeline 的优势是可控、可解释、可分阶段优化。

最现实的路线是 hybrid：用生成模型提升表示与召回补充，而不是一次性替换成熟在线系统。

公式 / 工程抓手

KEY	Future stack $\approx$ Efficient Retrieval + Scalable Ranking + Long-Sequence Modeling + Semantic/Generative Components
常见失分	回答“全面换大模型”，没有迁移路径、成本和失败保护。

连续追问

- 1. 如果只允许改一个阶段你改哪里？
- 2. 如何做 shadow/canary 迁移？
- 3. 未来推荐 foundation model 的训练数据是什么？

附录 A：把一道题答到 5 层的“追问树”

高质量面试不是背到第一层，而是能沿着“定义 → 为什么 → 公式/机制 → 工程权衡 → 线上验证”连续深入。下面以双塔为例：

Level 1	双塔是什么？——User Tower / Item Tower 独立编码，最后点积。
Level 2	为什么适合召回？——Item 向量可预计算并建立 ANN 索引。
Level 3	怎么训练？——InfoNCE / sampled softmax，核心在 negative distribution。
Level 4	In-batch 有什么偏差？——False Negative + popularity/sampling bias。
Level 5	离线 Recall 涨了为什么线上不涨？——候选增量、索引误差、后续排序吞噬、分布差异、时延。

项目拷打模板

- 背景：业务规模、用户/Item 数、QPS、当前瓶颈是什么？
- Baseline：为什么选它？原系统缺点是什么？
- 改动：数据、特征、模型、loss、采样、serving 分别改了什么？
- 实验：离线指标、A/B 流量、周期、显著性、护栏指标。
- 归因：收益到底来自模型结构还是数据/采样/算力？
- 失败：尝试过什么无效方案？为什么失败？
- 工程：Latency、显存、CPU/GPU、特征 freshness、故障回滚。

附录 B：7 天 / 30 天冲刺路线

如果时间有限，优先建立主线，再补模型细节。下面按“能应对面试”的收益排序。

周期	重点	每天输出
7 天 Day1	完整链路 + 项目复盘	画出自己的召回→排序→重排图，准备 3 分钟项目介绍
Day2	双塔 / InfoNCE / 负采样 / ANN	手写 loss，解释 false negative 与 hard negative
Day3	FM / DeepFM / DCNv2	手推 FM 二阶优化与 DCN cross
Day4	DIN / DIEN / SIM / Transformer	比较四者的目标、复杂度、长序列方案
Day5	MMoE / PLE / ESMM	画结构图，讲清 negative transfer 与 SSB
Day6	AUC / GAUC / A-B / Bias	用一个真实项目解释离线到线上
Day7	100 题模拟 + 前沿	只看题目口述；重点补 HSTU / RankMixer / Generative Rec
30 天	前 14 天系统学习；后 16 天项目深挖 + mock interview	每两天做一次 45 分钟模拟面试，记录被追问断点

最优先的 20 个节点

完整链路 → ItemCF → 双塔 → InfoNCE → 负采样 → ANN → DeepFM → DCNv2 → DIN → DIEN → SIM → Transformer → MMoE → PLE → ESMM → AUC/GAUC → A/B Test → Debias → HSTU → RankMixer。

参考资料与题源说明

公开面经具有个人记忆偏差，且并非公司官方题库。本手册对同义问法做了归并，并用原始论文补足模型定义。下列资料用于验证问题趋势与技术事实。

[I1] 美团推荐算法面经（2025-11）

Nowcoder · <https://www.nowcoder.com/feed/main/detail/87082efbdcfb4023b536c06ada8ce1a1>

[I2] 26 校招美团推荐算法一面（2025-12）

Nowcoder · <https://www.nowcoder.com/feed/main/detail/16e5b9c65159469983165ce7f37c1884>

[I3] 快手推荐算法一面（2025-09）

Nowcoder · <https://www.nowcoder.com/feed/main/detail/5db62c850f5c485bb0ec279b7be9b259>

[I4] 快手推荐算法实习一面面经

Nowcoder · <https://www.nowcoder.com/feed/main/detail/386ed2e833f44d299cdaf180d0257c06>

[I5] 美团搜推算法: PLE / DIN / In-batch 偏差

Nowcoder · <https://www.nowcoder.com/feed/main/detail/a6ca5f8234be43ff964f27535ec35ca0>

[P1] DeepFM: A Factorization-Machine based Neural Network for CTR Prediction

IJCAI 2017 · <https://www.ijcai.org/proceedings/2017/239>

[P2] Deep Interest Network for Click-Through Rate Prediction

KDD / arXiv · <https://arxiv.org/abs/1706.06978>

[P3] Deep Interest Evolution Network for CTR Prediction

AAAI / arXiv · <https://arxiv.org/abs/1809.03672>

[P4] Modeling Task Relationships in Multi-task Learning with Multi-gate Mixture-of-Experts

KDD 2018 / Google Research · <https://research.google/pubs/modeling-task-relationships-in-multi-task-learning-with-multi-gate-mixture-of-experts/>

[P5] Progressive Layered Extraction (PLE)

RecSys 2020 · <https://recsys.acm.org/recsys20/session-4/>

[P6] Entire Space Multi-Task Model (ESMM)

arXiv 2018 · <https://arxiv.org/abs/1804.07931>

[P7] DCN V2: Improved Deep & Cross Network

arXiv 2020 · <https://arxiv.org/abs/2008.13535>

[P8] BPR: Bayesian Personalized Ranking from Implicit Feedback

UAI 2009 / arXiv · <https://arxiv.org/abs/1205.2618>

[P9] Actions Speak Louder than Words: Trillion-Parameter Sequential Transducers for Generative Recommendations

ICML 2024 / PMLR · <https://proceedings.mlr.press/v235/zhai24a.html>

[P10] RankMixer: Scaling Up Ranking Models in Industrial Recommenders

arXiv 2025 · <https://arxiv.org/abs/2507.15551>

版本说明	本版聚焦推荐系统知识，不包含 LeetCode/概率统计/机器学习通用八股。题目截至 2026 年 9 月按公开资料与经典论文整理。
------	--